

SUPSI

Proxy basato su Appwrite

Studente/i

**Mazza Luca
Mondini Manuela**

Relatore

Brocco Amos

Correlatore

-

Committente

Brocco Amos

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11160

Anno

2025 / 2026

Data

8 maggio 2026

Indice

1	Abstract	1
2	Introduzione	3
2.1	Pianificazione	5
2.2	Requisiti	6
2.3	Conclusione	6
3	Stato dell'arte	7
3.1	Limiti delle infrastrutture di rete tradizionali	8
3.2	Tecnologie di superamento dei Firewall	8
3.3	Protocolli di messaggistica e middleware	8
3.4	Il paradigma Backend-as-a-Service (BaaS)	9
3.5	Il ruolo di Appwrite come middleware evoluto	9
3.6	WebAssembly e Qt	9
3.7	Comparativa delle tecnologie	9
3.8	Conclusione	10
4	Design e implementazione	11
4.1	Panoramica dell'architettura e del dominio	12
4.1.1	Struttura dei dati su Appwrite	12
4.2	Modello di dominio	13
4.2.1	Struttura del modulo	13
4.2.2	Entità principali	13
4.2.3	Serializzazione e parsing	13
4.2.4	Validazione	14
4.2.5	Gestione della sessione	14
4.2.6	Integrazione con gli altri componenti	14
4.3	Appwrite SDK	14
4.3.1	Componenti principali	15
4.3.2	Comunicazione asincrona (con signals e slots)	15
4.3.3	Gestione della sessione	15
4.3.4	Dualità Server-Client	15
4.3.5	Utilizzo	16
4.3.6	Gestione degli errori	16
4.4	Resilienza e controllo	16
4.4.1	Cache dei messaggi	17
4.4.2	Gestione della recovery	18
4.4.3	Controllo del Throughput	19
4.5	Gestione degli errori	20
4.5.1	Classificazione degli errori	20
4.5.2	Struttura dell'errore	21

4.5.3	Rilevazione e formattazione	21
4.5.4	Integrazione nel sistema	21
4.5.5	Scelte progettuali	21
4.6	Gestione dello stato del client	21
4.6.1	Struttura dello stato	22
4.6.2	Gestione dell'ordinamento e recovery	22
4.6.3	Contesto di comunicazione	22
4.6.4	Stato di autenticazione	22
4.6.5	Stato della connessione	22
4.6.6	Integrazione con gli altri componenti	23
4.6.7	Scelte progettuali	23
4.7	Elaborazione dei messaggi in ingresso	23
4.7.1	Ruolo del componente	23
4.7.2	Dipendenza dallo stato del client	24
4.7.3	Metodo processIncoming()	24
4.7.4	Deduplicazione	24
4.7.5	Rilevazione di gap	24
4.7.6	Aggiornamento dello stato	25
4.7.7	Reset del contesto	25
4.7.8	Scelte progettuali	25
4.8	Comunicazione in tempo reale	25
4.8.1	Architettura del modulo Realtime	25
4.8.2	Sottoscrizione ai canali	26
4.8.3	Gestione degli eventi	26
4.8.4	Gestione della connessione	26
4.8.5	Integrazione con il sistema	27
4.9	Pulizia automatica dei dati	27
4.9.1	Motivazione e contesto	27
4.9.2	Policy di persistenza	27
4.9.3	Algoritmo di pulizia	28
4.9.4	Gestione asincrona	28
4.9.5	Privilegi e sicurezza	28
4.9.6	Integrazione nel ciclo di vita del server	28
4.10	API per Server	29
4.10.1	Logica di Bootstrap	29
4.10.2	Gestione dei canali e della membership	30
4.10.3	Provisioning	30
4.11	API per Client	31
4.11.1	Descrizione generale	31
4.11.2	Componenti interni	31
4.11.3	Flusso operativo	31
4.11.4	Gestione dello stato	31
4.11.5	Gestione dell'autenticazione	32
4.11.6	Gestione del canale	32
4.11.7	Caricamento dei messaggi	32
4.11.8	Ricezione realtime	32
4.11.9	Invio dei messaggi	33
4.11.10	Gestione del recupero	33
4.11.11	Gestione degli errori	33
4.11.12	Limitazioni attuali	33
4.12	Conclusione	33

5	Proof of Concept	36
5.1	Backend	37
5.1.1	Requisiti di validazione	37
5.1.2	Architettura del sistema	37
5.1.3	Assegnazione del numero di sequenza	37
5.2	Frontend	38
5.2.1	Requisiti di validazione	38
5.2.2	Architettura del sistema	38
5.2.3	Flusso operativo del client	39
5.3	Porting WebAssembly	39
6	Test e validazione	40
6.1	Analisi delle suites di test	41
6.1.1	Moduli di dominio e consistenza dati	41
6.1.2	Moduli di controllo del flusso	41
6.1.3	Moduli applicativi	41
6.1.4	Integrazione con infrastruttura reale	41
6.2	Risultati	41
6.3	Conclusioni	42
7	Risultati	43
7.1	Valutazione architetturale	44
7.2	Validazione del Proof of Concept	44
7.3	Prestazioni e limiti riscontrati	44
7.4	Manuale d'uso	44
8	Conclusioni	45
8.1	Sintesi del lavoro	46
8.2	Sviluppi futuri	46
8.3	Considerazioni finali	47
	Glossario	I
	Sitografia	IV
A	Guida utente - Backend	VI
A.1	Prerequisiti	VI
A.2	Configurazione iniziale	VI
A.3	Avvio del servizio	VI
A.4	Comandi amministrativi	VII
A.4.1	Utenti	VII
A.4.2	Canali	VII
A.4.3	Membri del canale	VII
A.4.4	Sessioni	VII
A.4.5	Messaggi	VII
A.5	Flusso operativo consigliato	VIII
A.6	Output e gestione errori	VIII
B	Guida utente - Frontend	X
B.1	Prerequisiti	X
B.2	Configurazione iniziale	X
B.3	Avvio dell'applicazione	XI
B.4	Schermata iniziale	XI

B.5 Login tramite email	XII
B.6 Login come guest	XII
B.7 Utilizzo della chat	XII
B.8 Invio dei messaggi	XII
B.9 Ricezione dei messaggi	XIII
B.10 Logout	XIII
B.11 Errori comuni	XIII
B.12 Flusso operativo consigliato	XIII
B.13 Avvio della versione WebAssembly	XIV

Elenco delle figure

4.1	Interazioni tra Client, Server e Appwrite	12
4.2	Struttura di classi di AppwriteSDK	14
4.3	Struttura di RecentMessageCache	17
4.4	Struttura di RecoveryManager	19
4.5	Struttura di RateLimiter	20
4.6	Struttura di Realtime	26
4.7	Struttura di GarbageCollector	27
4.8	Lifecycle di AppcommServer	29
4.9	UML Sequence, processo di Bootstrap	29
4.10	Sequence Diagram UML inizializzazione e sincronizzazione Client.	35
5.1	Sequence Diagram del sistema Inbox/Outbox	38

Elenco delle tabelle

2.1	Tecnologie utilizzate nel progetto	4
2.2	Requisiti di Appcomm	6
3.1	Analisi comparativa delle tecnologie	9
4.1	Codici di errore	20
4.2	Stati di connessione	23
5.1	Requisiti di validazione Backend	37
5.2	Requisiti di validazione Frontend	38
6.1	Risultati delle test suites	42

Capitolo 1

Abstract

Lo sviluppo di applicazioni distribuite moderne richiede sistemi di comunicazione in tempo reale affidabili e reattivi. Tuttavia, la gestione delle instabilità di rete e la sincronizzazione dello stato tra client e server introducono notevoli complessità architetturali.

Questo documento presenta la progettazione e l'implementazione di appcomm, una libreria C++ nativa basata sul framework Qt [1], ideata per astrarre e rendere resiliente la comunicazione tra applicazioni frontend e l'infrastruttura Backend-as-a-Service Appwrite¹ [2].

L'architettura proposta si articola in tre macro-componenti: un wrapper SDK ottimizzato per l'integrazione asincrona, un ecosistema di resilienza client-side, un gestore di recupero per il riallineamento automatico e un limitatore di traffico e un modulo server-side orchestratore dedicato al setup automatizzato dell'infrastruttura, alla gestione dell'amministrazione del sistema.

L'efficacia della libreria e delle scelte architetturali è stata validata attraverso lo sviluppo di un Proof of Concept completo. I risultati dimostrano come l'integrazione di questi moduli consenta alla libreria di auto-riparare lo stato locale dopo disconnessioni temporanee, garantendo la coerenza globale dei messaggi e offrendo agli sviluppatori un'interfaccia di comunicazione robusta, scalabile e fault-tolerant.

Developing modern distributed applications requires reliable and responsive real-time communication systems. However, managing network instabilities, synchronizing state between client and server, and secure infrastructure provisioning introduces architectural complexity.

This paper presents the design and implementation of appcomm, a native C++ library based on the Qt framework [1], designed to abstract and resilient communication between frontend applications and the Appwrite [2] Backend-as-a-Service infrastructure.

The proposed architecture is divided into three macro-components: an SDK wrapper optimized for asynchronous integration, a client-side resilience ecosystem, a manager for automatic realignment, and a server-side orchestrator module dedicated to automated infrastructure setup and management.

The effectiveness of the library and its architectural choices was validated through the development of a comprehensive Proof of Concept. The results demonstrate how the integration of these modules allows the library to self-repair local state after temporary disconnections, ensuring global message consistency and offering developers a robust, scalable, and fault-tolerant communication interface.

¹<https://appwrite.io>

Capitolo 2

Introduzione

Il progetto nasce dall'esigenza di risolvere una delle sfide che più comunemente affligge le architetture distribuite moderne, ovvero la comunicazione bidirezionale asincrona tra un client ed un server che risiedono in reti private.

Lo scopo del progetto è quindi lo sviluppo di una libreria C++ che permetta di utilizzare Appwrite come middleware (proxy) per facilitare la creazione di canali di comunicazione sicuri e scalabili tra client e server. Al posto di tentare connessioni dirette instabili o poco sicure, entrambe le entità si connettono ad un istanza Appwrite che funge da intermediario autenticato per lo scambio di messaggi strutturati in tempo reale. Il sistema deve garantire la sicurezza, fornendo accesso solamente ad utenti autenticati e autorizzati per l'accesso ai canali di comunicazione, delegando la gestione di credenziali, sicurezza e persistenza all'infrastruttura sottostante di Appwrite.

Gli obiettivi principali del lavoro sono articolati in diverse aree chiave:

- **Progettazione di un architettura sicura:** è necessario definire un modello di comunicazione dove il server agisca da amministratore dell'infrastruttura di comunicazione (essendo autenticato tramite API Key) ed il frontend come utente finale (autenticandosi come utente "normale"). Il server è inoltre responsabile della gestione dei canali di comunicazione, fornendo un meccanismo di creazione, lettura, aggiornamento e cancellazione (CRUD) per canali, sessioni, mentre invece il frontend si preoccupa semplicemente di connettersi a canali esistenti, inviare messaggi e ricevere notifiche di nuovi messaggi.
- **Sviluppo della libreria appcomm:** una libreria modulare, scritta in C++ e basata su Qt, che fornisca un'interfaccia semplice ed intuitiva per interagire con l'API di Appwrite è necessaria per la strutturazione del progetto; questa deve includere funzionalità per la gestione delle sessioni, l'identificazione dei canali basandosi su UUID, la serializzazione, validazione e impacchettamento dei payload e la gestione degli eventi di comunicazione in modo asincrono.
- **Proof of Concept:** per validare l'efficacia e il funzionamento corretto del sistema è necessario lo sviluppo di una semplice applicazione che dimostri la maniera di utilizzo della libreria in un contesto reale ed il funzionamento tecnico del meccanismo di comunicazione. Questa consiste in una chat semplice, dove un frontend, costruito con Qt/QML e compilato con WebAssembly per l'esecuzione tramite web browser, comunica con un servizio backend nativo in C++, attraverso i canali di comunicazione gestiti da Appwrite tramite appcomm. Per semplificare la dimostrazione, il backend si limita al ruolo di *echo server*.
- **Bootstrap deploy:** per facilitare l'adozione dell'intermediario Appwrite in soluzioni esistenti, è necessario che, in presenza di un istanza "pulita" di Appwrite ed il server, quest'ultimo sia in grado, solamente tramite la chiave API e l'URL dell'istanza, di creare in autonomia l'infrastruttura necessaria per il funzionamento, dimodoché sia possibile integrare Appwrite sia come "primo cambiamento" in un'architettura esistente, sia nel caso si necessiti di scalare orizzontalmente un sistema già configurato con questo proxy, permettendo di cambiare istanza in modo dinamico, senza necessità di riconfigurazione.

Il prodotto si avvale di uno stack tecnologico moderno e cross-platform, con l'obiettivo di massimizzare la portabilità e l'integrazione in diversi contesti applicativi, con un focus particolare su soluzioni web-based grazie alla compilazione in WebAssembly.

Tecnologia	Ruolo
Appwrite	Funge da middleware (proxy) per la comunicazione tra client e server
C++	Linguaggio di programmazione per la libreria e il backend
Qt	Framework per lo sviluppo della libreria e del frontend
QML	Linguaggio per la definizione dell'interfaccia utente del frontend
WebAssembly	Compilazione del frontend per l'esecuzione in browser
JSON	Formato per la serializzazione dei messaggi scambiati tra client e server
Docker	Contenitore per il deploy dell'istanza Appwrite e del backend

Tabella 2.1: Tecnologie utilizzate nel progetto

2.1 Pianificazione

Di seguito sono descritte le fasi principali che sono state pianificate per lo sviluppo del progetto. Il piano non è ovviamente rigido, data la natura iterativa dello sviluppo software, ma fornisce una roadmap generale per l'implementazione e la validazione del sistema.

La presenza di un piano che non sia troppo statico è fondamentale per valorizzare l'obiettivo di sviluppo piuttosto che la semplice realizzazione di un prodotto, permettendo di adeguarsi dinamicamente alle esigenze che potenzialmente emergono durante le varie fasi di sviluppo e di discussione con il relatore.

Essendo anche le tecnologie usate relativamente nuove per tutti i membri del team, soprattutto per quanto riguarda Appwrite e WebAssembly, è stato necessario prevedere una fase iniziale di analisi e studio approfondito, prima di procedere con l'implementazione vera e propria, per garantire una comprensione solida dei meccanismi da implementare.

1. **Analisi:** occorre dello studio dedicato ad approfondire come possa essere utilizzato Appwrite come intermediario per la comunicazione tra client e server, comprendere come adattare l'API dello stesso alle esigenze tecniche del progetto e studiare le tecnologie coinvolte;
2. **Pianificazione:** è necessario definire un piano di sviluppo per quanto riguarda la libreria appcomm, identificando le funzionalità chiave da implementare, formalizzandole tramite delle task. Questo va ripetuto anche per la parte di sviluppo del frontend e del backend;
3. **Design:** è necessario definire un'architettura per la libreria appcomm che sia modulare, scalabile e manutenibile, identificando i moduli chiave, le loro responsabilità e le interazioni tra di essi. Inoltre, è necessario definire un'architettura generale per il sistema, identificando i componenti chiave, le loro responsabilità e le interazioni tra di essi. Ciò include la produzione di diversi diagrammi che al meglio esplichino le scelte architettureali, come ad esempio diagrammi delle classi, diagrammi di sequenza e diagrammi di flusso;
4. **Implementazione Libreria:** occorre implementare dapprima la libreria appcomm, la quale verrà implementata con il classico approccio per task, dove si analizza dapprima il relativo requisito, si implementa quanto necessario per soddisfarlo, si definisce un'unità di test per validarne il funzionamento e si procede con quanto segue. Lo sviluppo della libreria deve essere imperativamente preceduto da una fase di design dell'architettura della stessa, per garantire una struttura modulare, scalabile e manutenibile;
5. **Implementazione PoC:** una volta che la libreria è sufficientemente matura, è possibile procedere con lo sviluppo del Proof of Concept, il quale ha l'unico scopo di validare il funzionamento tecnico del sistema, deve quindi essere il più semplice possibile, limitato appunto alla dimostrazione del funzionamento della libreria;
6. **Test e Validazione:** una volta che il PoC è completo, è necessario procedere con una fase di test e validazione del prodotto finito, per garantire che tutte le funzionalità siano implementate correttamente e che il sistema funzioni come previsto, dal punto di vista del client e da quello del server, in scenari di comunicazione reali;
7. **Documentazione:** parallelamente alle fasi di sviluppo, è necessario procedere con la stesura della documentazione tecnica del progetto, che include sia la documentazione della libreria appcomm, sia un manuale d'uso per il prodotto finito, che spieghi come utilizzare il sistema, come configurarlo e come integrarlo in soluzioni esistenti. Questa guida utente riassume la documentazione tecnica integrata nei vari moduli della libreria, fornendo esempi pratici di utilizzo e consigli di troubleshooting per facilitare l'adozione del sistema.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

2.2 Requisiti

ID	Descrizione	Note
R-01	<i>Appcomm</i> deve astrarre l'API di Appwrite	Modulo dedicato
R-02	<i>Appcomm</i> deve esporre lo stato di connessione con Appwrite	-
R-03	<i>Appcomm</i> detiene dei modelli di dominio	-
R-04	<i>Appcomm</i> deve filtrare i messaggi sulla base del destinatario	-
R-05	<i>Appcomm</i> necessita un sistema di Recovery per i messaggi persi	-
R-06	<i>Appcomm</i> tiene una cache degli ultimi n messaggi	-
R-07	<i>Appcomm</i> immagazzina una policy di persistenza dei messaggi	-
R-08	<i>Appcomm</i> gestisce un modello unificato per gli errori	-
R-09	<i>Appcomm</i> rende intercambiabile la modalità di comunicazione (RT/- Poll)	-
R-10	<i>Appcomm</i> fornisce un Wrapper per la gestione di connessione/eventi	-
R-11	<i>Appcomm</i> fornisce dei servizi per le query veloci	-
R-12	<i>Appcomm</i> fornisce sistema di Garbage Collection di vecchi messaggi	- /

Tabella 2.2: Requisiti di Appcomm

2.3 Conclusione

Questo capitolo ha definito il perimetro del progetto, chiarendo il problema da risolvere, il valore tecnico della soluzione ed i criteri con cui valutarne l'efficacia. In particolare, l'utilizzo di Appwrite come middleware, consente di spostare la complessità infrastrutturale verso un livello gestito, mantenendo il controllo della logica applicativa nelle implementazioni di backend in C++ e la qualità del software.

La pianificazione proposta non rappresenta un semplice elenco di attività, ma una strategia incrementale che collega lo studio delle tecnologie, la progettazione architettuale, l'implementazione per task e la validazione continua. Questo approccio riduce il rischio tecnico, favorisce la tracciabilità delle decisioni e rende il progetto adattabile a soluzioni scaturenti da evoluzioni future, senza comprometterne la coerenza complessiva.

Infine, i requisiti raccolti costituiscono il contratto funzionale dell'intera soluzione; ogni modulo introdotto nei capitoli successivi risponde direttamente ad almeno una delle esigenze esplicitate durante questa fase. In questo senso l'introduzione non è solo un contesto descrittivo, ma la base di riferimento che guida il design, implementazione e validazione del sistema end-to-end.

Capitolo 3

Stato dell'arte

L'interconnessione tra sistemi residenti in reti locali protette e interfacce web rappresenta una delle sfide architetturali più rilevanti nei sistemi distribuiti moderni. Il problema fondamentale risiede nella natura asimmetrica dei sistemi NAT e Firewall, che permettono, tipicamente senza particolari filtri, le connessioni Outbound ma bloccano sistematicamente i tentativi di accesso Inbound.

Questo capitolo inquadra il progetto rispetto alle soluzioni già disponibili, evidenziandone i limiti, i trade-off e le opportunità delle principali strategie impiegate per superare le barriere di rete. L'obiettivo non è soltanto presentare una rassegna tecnica, ma costruire un criterio di confronto utile a motivare la scelta architetturale adottata nel lavoro; un modello client-server mediato da una piattaforma BaaS e supportato da componenti nativi C++ e WebAssembly.

L'analisi considera approcci infrastrutturali classici, protocolli orientati alla messaggistica e servizi cloud moderni, con attenzione agli aspetti che impattano direttamente il caso d'uso del progetto: sicurezza operativa, portabilità cross-platform, complessità di integrazione e sostenibilità di manutenzione. In questa prospettiva, il capitolo funge da ponte tra il problema introduttivo e le scelte di design che verranno formalizzate nel capitolo successivo.

3.1 Limiti delle infrastrutture di rete tradizionali

La maggior parte delle reti locali opera dietro configurazioni NAT e Firewall Stateful. Mentre queste tecnologie proteggono la rete interna, impediscono l'instaurazione di socket passivi all'interno della LAN, che siano raggiungibili dall'esterno.

Le soluzioni storiche, come il Port Forwarding o UPnP sono oggi considerate deprecate in contesti professionali a causa dell'elevata vulnerabilità ai vettori d'attacco *Zero-day* e della dipendenza verso hardware di rete, che non garantisce la portabilità della soluzione.

3.2 Tecnologie di superamento dei Firewall

Attualmente, le metodologie per consentire la comunicazione tra entità separate da barriere di rete si dividono in tre categorie principali, che rappresentano le soluzioni più *tradizionali*; ognuna di queste ha dei limiti operativi specifici:

- **Port Forwarding & Reverse Proxy:** Queste tecnologie prevedono l'apertura statica di porte direttamente sui router, oppure di esporre alla rete WAN un server che risiede invece nella LAN. Sebbene possano essere efficienti, aumentano la superficie di attacco del backend e richiedono privilegi di amministrazione sulla rete non sempre disponibili in contesti aziendali o sotto reti mobili.
- **VPN (Virtual Private Network):** Consiste nella creazione di un tunnel logico attraverso la rete, nel quale è possibile comunicare direttamente in maniera sicura. Queste tuttavia introducono un importante overhead in termini di latenza, e spesso richiedono l'installazione di software specifici per il client, rendendole inadatte per applicazioni web-based leggere o temporanee.
- **Protocolli Pub/Sub (MQTT¹/AMQP²):** Utilizzano un broker centrale come punto di incontro. Sebbene ideali per l'IoT, la loro integrazione in ambienti Web puro richiede spesso gateway WebSocket aggiuntivi, complicando il deployment in architetture cross-platform.

3.3 Protocolli di messaggistica e middleware

Esistono diverse tecnologie che tentano di risolvere il problema del *punto di incontro* tra due entità separate.

Tra queste appare REST, il quale è de-facto lo standard per il web. Tuttavia REST è per natura *stateless*, ed è basato su un modello request-response. Per ottenere la bidirezionalità necessaria ad un sistema di controllo o ad una chat, richiede tecniche di Long Polling³, che risultano inefficienti in termini di banda e latenza.

Un'altra tecnologia è WebRTC, il quale è un protocollo avanzato, mirato alla comunicazione Peer to Peer. Sebbene offra latenze minime, la sua implementazione in C++ nativo richiede librerie di enormi dimensioni ed è estremamente complessa da integrare in un sistema Wasm senza un server di segnalazione esterno.

Infine è anche presente MQTT, il quale è invece un protocollo leggero di tipo Pub/Sub. È molto efficace, ma la sicurezza è spesso delegata a strati esterni, e non offre una persistenza dei dati strutturata.

¹<https://mqtt.org/>

²<https://amqp.org/>

³<https://ably.com/topic/long-polling>

3.4 Il paradigma Backend-as-a-Service (BaaS)

L'evoluzione dei servizi Cloud ha portato all'emergere di piattaforme BaaS⁴ (come Firebase, Supabase o Appwrite). Queste soluzioni agiscono come un middleware neutro sulla rete pubblica. Il vantaggio risiede nel fatto che sia il client che il server effettuano connessioni out-bound verso la piattaforma, che sono permesse dai firewall standard, eliminando la necessità di esporre porte fisiche sul backend nativo.

3.5 Il ruolo di Appwrite come middleware evoluto

Appwrite si inserisce in questo scenario non solo come un database, ma come un Orchestratore di Backend. A differenza di un semplice broker di messaggi, Appwrite fornisce:

- **Identity Management:** L'integrazione nativa di sessioni utente (JWT o Cookie) permette di proteggere i canali di comunicazione senza dover implementare logiche di cifratura custom.
- **Protocollo Realtime:** Appwrite astrae la gestione dei socket. Il client si sottoscrive a un canale (documento) e riceve notifiche push al variare dello stato, simulando una comunicazione diretta pur passando per un proxy sicuro.
- **Self-Hosting/Cloud:** Seguendo la filosofia dell'indipendenza tecnologica, il supporto a Docker permette di evitare il Vendor Lock-in, tipico di piattaforme come Firebase.

3.6 WebAssembly e Qt

L'aspetto che porta l'innovazione più grande del progetto è la possibilità di utilizzare WebAssembly per compilare applicazioni Qt, per poi essere in grado di utilizzarle tramite Web Browser. Ciò permette di sviluppare una libreria unica, che funzioni per il backend in C++, e che sia compatibile con l'applicazione web del frontend.

In tale situazione, è garantita la serializzazione e deserializzazione identica dei messaggi, la gestione identica degli errori e che i test di unità siano validi per l'intero ecosistema.

3.7 Comparativa delle tecnologie

Categoria	Soluzione	Persistenza	Sicurezza
Direct	Port Forwarding	No	Bassa
Tunnel	VPN/SSH	No	Alta
Pub/Sub	MQTT	Limitata	Media
BaaS Proxy	Appwrite	Alta	Alta

Tabella 3.1: Analisi comparativa delle tecnologie

⁴<https://geeksforgeeks.org/firebase/what-is-firebase/>

3.8 Conclusione

In conclusione, la soluzione proposta rappresenta un avanzamento rispetto ai metodi tradizionali di comunicazione cross-network. Sfruttando la robustezza di un middleware BaaS e la portabilità del binario WebAssembly, il progetto risolve i problemi di latenza, sicurezza e configurazione di rete in un'unica architettura software modulare e mantenibile.

L'analisi comparativa mostra che le alternative più comuni tendono a massimizzare solo una parte del problema; alcune offrono bassa latenza ma richiedono infrastrutture complesse, altre semplificano il deployment ma non garantiscono persistenza strutturata o la sicurezza integrata. L'approccio basato su Appwrite invece, combina autenticazione, persistenza del documento e canali realtime in un unico punto di integrazione, riducendo la quantità di componenti specializzati da mantenere.

Ne consegue che il contributo principale non è la semplice adozione di una tecnologia cloud, ma la definizione di una nuova composizione architetturale coerente con i vincoli reali del progetto; le reti private, il frontend web, il backend nativo e le necessità di evoluzione progressiva. Questa conclusione giustifica il passaggio al capitolo di design, dove le scelte teoriche qui discusse vengono tradotte in moduli software, responsabilità ben separate e flussi operativi verificabili.

Capitolo 4

Design e implementazione

Questo capitolo descrive come i requisiti e le valutazioni tecnologiche precedenti siano trasformati in una struttura software concreta, con particolare attenzione alla separazione delle responsabilità, alla robustezza operativa e alla manetunibilità del codice nel medio periodo. L'obiettivo è rendere esplicita la corrispondenza tra i problemi architetturali identificati e le soluzioni implementative adottate nei moduli della libreria appcomm.

La trattazione segue un percorso progressivo; dapprima il modello di dominio e l'SDK di accesso ad Appwrite, poi i componenti di resilienza, il modello di errore, la gestione dello stato del client, la pipeline di elaborazione dei messaggi, la comunicazione realtime ed infine le API client e server side. In questo modo il capitolo documenta sia le scelte di struttura sia i meccanismi di comportamento che garantiscono consistenza del protocollo, recupero di anomalie e del controllo di carico.

La finalità è quella di fornire una base tecnica sufficientemente dettagliata da consentire l'estensione, revisione e integrazione in contesti applicativi differenti, mantenendo però un'architettura unitaria tra frontend e backend.

4.1 Panoramica dell'architettura e del dominio

Prima di analizzare i singoli moduli, è fondamentale definire la struttura ad alto livello del sistema. L'architettura si divide in tre attori principali:

- **Frontend (Client):** L'applicazione utente (compilata in WebAssembly) che utilizza `AppcommClient` per inviare e ricevere messaggi e per mantenere sincronizzata la chat.
- **Backend Appwrite:** L'infrastruttura BaaS che funge da proxy e database centrale. Gestisce le connessioni, i permessi, gli eventi in tempo reale e lo storage persistente.
- **Backend (Server):** Un servizio backend nativo che utilizza `AppcommServer` (con privilegi amministrativi) per il provisioning dell'infrastruttura, il routing avanzato e operazioni di manutenzione come la Garbage Collection.

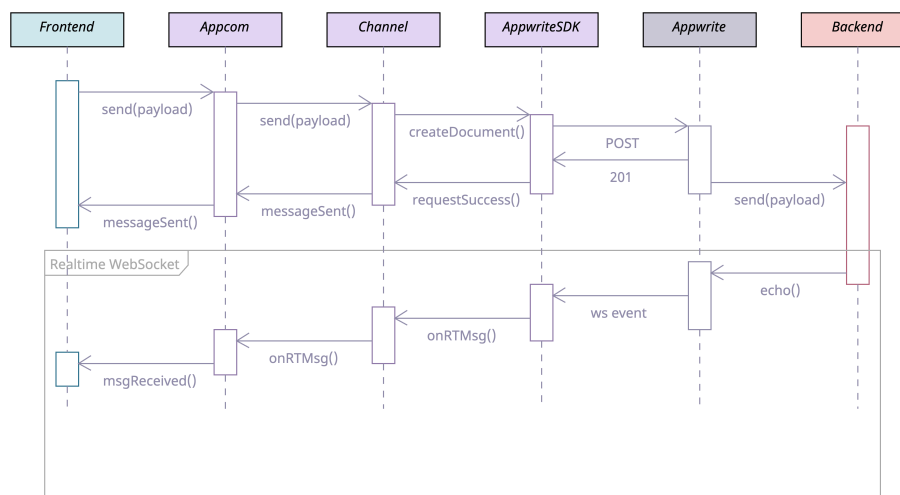


Figura 4.1: Interazioni tra Client, Server e Appwrite

4.1.1 Struttura dei dati su Appwrite

Per supportare la logica di dominio, in fase di provisioning il Server genera un Database dedicato su Appwrite, all'interno del quale crea quattro Collezioni principali:

1. **channels:** Ogni documento in questa collezione rappresenta un "Canale" logico di comunicazione (es. una stanza di chat), contenente metadati come il nome e la data di creazione.
2. **messages:** Contiene tutti i messaggi scambiati. Ogni documento è logicamente associato a un `topicId` e contiene il `payload`, il mittente e un `sequenceNumber` per l'ordinamento globale.
3. **members:** Mappa l'associazione multi-a-uno tra utenti e canali, gestendo lo stato di presenza e di attività.
4. **sessions:** Gestisce il ciclo di vita logico delle sessioni temporanee di comunicazione.

4.2 Modello di dominio

Per disaccoppiare la logica applicativa dalla rappresentazione JSON dei dati, il progetto introduce un modulo di dominio centralizzato definito nel file `model.h`. Questo modulo raccoglie tutte le strutture dati utilizzate nello scambio di informazioni tra frontend, proxy e backend.

L'obiettivo è fornire una rappresentazione tipizzata e coerente delle entità del sistema, evitando la manipolazione diretta di oggetti `JsonObject` nei livelli superiori.

4.2.1 Struttura del modulo

Tutte le entità sono definite all'interno del namespace `model`, che rappresenta il punto di accesso unico per il dominio applicativo. Le strutture sono progettate per essere in primis leggere e facilmente serializzabili, indipendenti dal trasporto di dati e riutilizzabili tra i diversi moduli del sistema.

4.2.2 Entità principali

Il modello definisce diverse entità fondamentali:

- **User**: rappresenta un utente tramite identificatore e email, con metodo `isValid()` per verificare la consistenza dei dati;
- **Topic**: identifica un canale di comunicazione tramite un ID e un nome visibile in UI;
- **Message**: rappresenta l'unità base della comunicazione. Contiene un identificatore del canale, un identificatore del mittente, uno per il messaggio, un timestamp in formato UTC, un payload generico in formato JSON ed un flag `isEcho` per distinguere messaggi generati localmente da quelli ricevuti dal server.
- **SessionInfo**: contiene metadati relativi alla sessione, inclusi tipo di autenticazione e timestamp di creazione e scadenza;
- **TopicMember**: descrive la presenza di un utente in un canale, includendo stato di attività e informazioni temporali;
- **PersistencePolicy**: definisce le politiche di persistenza (TTL) per messaggi, sessioni e canali inattivi;
- **AppcommConfig**: contiene i parametri di configurazione necessari per connettersi al backend Appwrite.

4.2.3 Serializzazione e parsing

Le entità del modello implementano metodi di conversione bidirezionale tra rappresentazione tipizzata e formato JSON. In particolare `fromJson()` consente di costruire oggetti a partire dai payload ricevuti dal backend mentre `toJson()` permette la serializzazione per l'invio dei dati.

Questo approccio centralizza la logica di parsing, evitando duplicazioni e garantendo coerenza tra i diversi componenti del sistema.

4.2.4 Validazione

Le strutture principali includono metodi di validazione come `isValid()`, utilizzati per verificare la presenza e la correttezza dei campi obbligatori prima dell'utilizzo degli oggetti.

Nel caso di parsing da JSON, se i dati non risultano validi, viene restituita un'istanza vuota. Questo consente ai moduli superiori di gestire esplicitamente i casi di errore senza propagare dati inconsistenti.

4.2.5 Gestione della sessione

Il tipo `SessionInfo` include il metodo `isExpired()`, che permette di verificare in modo immediato la validità temporale della sessione. Il tipo di autenticazione è modellato tramite l'enumerazione `AuthType`, che distingue tra sessioni anonime (guest) e autenticazione tramite email.

4.2.6 Integrazione con gli altri componenti

Il modello di dominio è utilizzato trasversalmente dai principali moduli del sistema; l'SDK riceve dati in formato JSON dal backend, il `RecoveryManager` converte tali dati in oggetti `model::Message`, la `RecentMessageCache` memorizza solo messaggi validati ed i livelli applicativi lavorano esclusivamente con strutture tipizzate.

Questa architettura garantisce una separazione chiara tra livello di trasporto e logica applicativa, migliorando robustezza, leggibilità e manutenibilità del codice.

4.3 Appwrite SDK

Il modulo `AppwriteSDK` costituisce il cuore tecnologico della libreria `appcomm`. Questo astrae l'API REST¹ di Appwrite, permettendo al resto della libreria di interagire con il backend senza dover gestire manualmente la complessità del protocollo HTTP o della gestione delle sessioni.

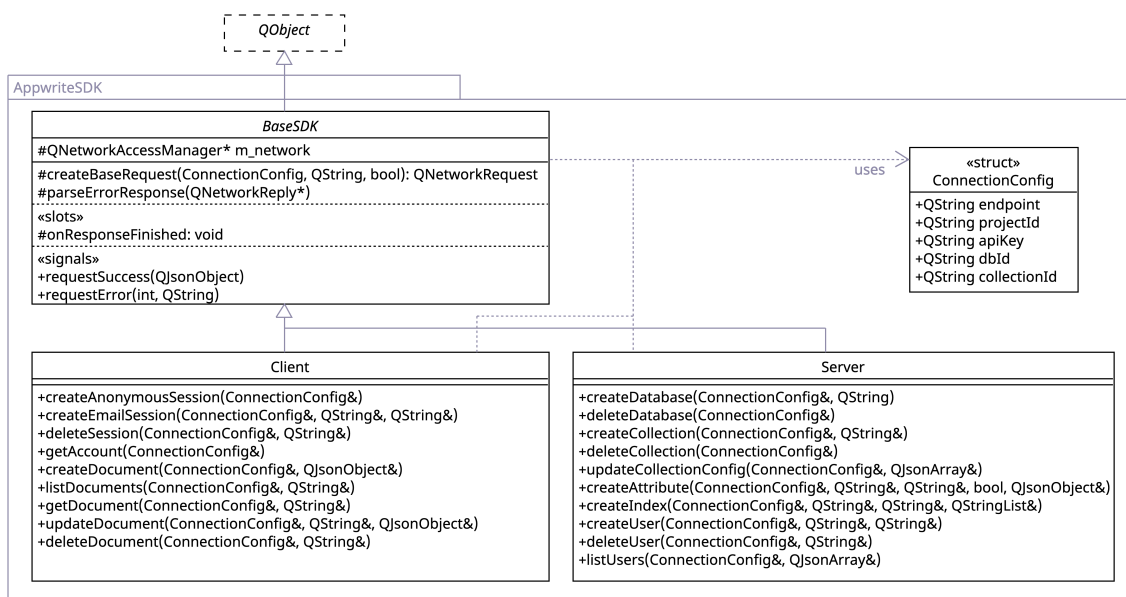


Figura 4.2: Struttura di classi di AppwriteSDK

¹<https://appwrite.io/docs/apis/rest>

L'architettura del modulo segue i principi di ereditarietà, con lo scopo principale di separare le responsabilità per quello che riguarda la parte che interessa al client, e quella che interessa al server.

La scelta di utilizzare Qt, tramite `QNetworkAccessManager`² è principalmente dovuta alla necessità del frontend di eseguire una build con WebAssembly, il quale non supporta librerie di comunicazione come `Curl`. Inoltre l'utilizzo di tale libreria, permette la connessione e la comunicazione tramite `signals` e `slots`, il che risulta consistente e comodo per il resto dell'implementazione e soprattutto per il testing del modulo.

4.3.1 Componenti principali

1. `ConnectionConfig`: Struttura che incapsula le credenziali e gli identificatori necessari per la comunicazione;
2. `BaseSDK`: Classe base astratta che gestisce l'istanza di `QNetworkAccessManager`, la gestione dei Cookie (con `QNetworkCookieJar`³) e la logica comune di parsing delle risposte e degli errori. Qui vengono impiegati `signals` e `slots` per la comunicazione.
3. `Client`: Specializzazione dedicata al frontend. Gestisce tutte le operazioni non-privilegiate, come l'autenticazione anonima o tramite email e la creazione di documenti nel database.
4. `Server`: Specializzazione privilegiata per il backend. Utilizzando la chiave API esegue le operazioni amministrative, come il provisioning del database, la creazione di attributes e la gestione degli utenti.

4.3.2 Comunicazione asincrona (con `signals` e `slots`)

A differenza di un approccio sincrono, quindi bloccante, l'utilizzo del meccanismo di Qt `signals` e `slots` è stato scelto per garantire la latenza minima di rete. Quando viene inviata una richiesta alla REST API, il controllo torna immediatamente all'applicazione; una volta ricevuta una risposta, viene emesso un segnale, tra `requestSuccess` o `requestError`. Tutto ciò è essenziale per WebAssembly, del quale il thread principale non può mai essere bloccato.

4.3.3 Gestione della sessione

L'SDK è configurato per utilizzare un `QNetworkCookieJar`. Nel caso del client, permette ad Appwrite di iniettare un cookie di sessione dopo il login. Qt gestisce automaticamente la persistenza di quest'ultimo nelle richieste successive, simulando lo stato di autenticazione tipico di un browser senza dover gestire manualmente i token JWT nel codice.

4.3.4 Dualità Server-Client

Sebbene il codice sia condiviso, l'SDK implementa una distinzione netta tra la modalità ADMIN e USER.

Il server agisce come supervisore; crea l'infrastruttura e gestisce l'integrità dei dati. Il client invece interagisce esclusivamente con i propri canali autorizzati. Questa separazione garantisce che le chiavi API amministrative non vengano mai compilate nel binario esposto pubblicamente.

²<https://doc.qt.io/qt-6/qnetworkaccessmanager.html>

³<https://doc.qt.io/qt-6/qnetworkaccessmanager.html>

4.3.5 Utilizzo

Per utilizzare il modulo, è dapprima necessario definire una `ConnectionConfig` e istanziare uno dei due componenti desiderati (`Client/Server`):

```
AppwriteSDK::ConnectionConfig config;
config.endpoint = "https://cloud.appwrite.io/v1";
config.projectId = "my-project-id";
config.dbId = "chat-db";
// Altri campi...

QNetworkAccessManager* mgr = new QNetworkAccessManager(this);
AppwriteSDK::Client* client = new AppwriteSDK::Client(mgr, this);
```

Di seguito un esempio di flusso, dove viene effettuato un login e l'invio di un messaggio:

```
// Connessione ai segnali
connect(client &AppwriteSDK::Client::requestSuccess, this,
[] (const QJsonObject& data) {
    qDebug() << "Success: " << data;
});

// Login Asincrono
client->createEmailSession(config, "user@example.com", "password123");

// Invio messaggio
QJsonObject msg;
msg["payload"] = "Hello, World!";
msg["topicId"] = "room-01";
client->createDocument(config, msg);
```

4.3.6 Gestione degli errori

Incluso nell'SDK, è presente un parser intelligente, che estrae i messaggi di errore restituiti da Appwrite (per esempio *Invalid credentials* o *Document not found* e li propaga attraverso il segnale `requestError`, permettendo una gestione centralizzata dei guasti nel modulo superiore di logica di business.

4.4 Resilienza e controllo

Oltre alla gestione del trasporto dati tramite l'SDK, la libreria `appcomm` integra dei componenti specializzati per garantire la continuità del servizio e la coerenza di stato tra client e server. Nello specifico, questi moduli mitigano tre tipologie di problemi critici nei sistemi distribuiti:

- **Perdita di messaggi per instabilità di rete:** Una caduta temporanea della connessione causerebbe la perdita di eventi push. Il sistema mitiga questo problema tracciando l'ultimo messaggio e richiedendo attivamente il "delta" mancante alla riconnessione.
- **Latenza e blocchi della UI:** L'attesa di scaricare i dati dal server ad ogni ricaricamento degraderebbe l'esperienza utente. Il buffer locale espone immediatamente i dati recenti mentre il layer di rete si sincronizza in background.
- **Esaurimento delle quote API (Flood):** Fasi di riconnessione o invii massivi potrebbero far scattare i blocchi anti-spam di Appwrite (Error 429). Il sistema scaglionava le richieste tramite controllo di flusso.

Questo garantisce che la logica di business sia separata dal trasporto, permettendo la gestione in modo robusto delle interruzioni.

Questo garantisce che la logica di business sia separata dal trasporto, permettendo la gestione in modo robusto delle interruzioni di rete e dei limiti infrastrutturali.

4.4.1 Cache dei messaggi

È quindi presente un modulo, RecentMessageCache, il quale è responsabile della gestione di un buffer di memoria volatile che conserva i messaggi transitati più di recente. La sua funzione principale è ridurre la latenza d'accesso ai dati recenti e fornire una base per le operazioni quando è necessario recuperare un'eventuale perdita di messaggi.

RecentMessageCache
-int m_capacity -QQueue<QString> m_queue -QHash<QString, Message> m_cache
+addMessage(Message&) +getMessagesSince(const QString&) -> QList<Message> +getMessage(const QString&) -> Message +getAllMessages() -> QList<Message> +contains(const QString&) -> bool +clear() +size() -> int +capacity() -> int +isEmpty() -> bool

Figura 4.3: Struttura di RecentMessageCache

Algoritmo di *Eviction*

La cache è implementata seguendo una politica LRU semplificata. Per massimizzare le prestazioni, vengono utilizzate due strutture dati coordinate. La scelta di utilizzare l'accoppiata Hash+Queue rispetto alla classe nativa QCache è dettata da motivazioni di ownership e controllo: QCache è progettata per gestire puntatori a oggetti e applicare sfratti basati sul "costo" di memoria. Lavorando invece con oggetti `model::Message` estremamente leggeri e condivisi implicitamente da Qt (Implicit Sharing), una combinazione esplicita risulta più efficiente e permette un controllo rigido e prevedibile dell'ordinamento strettamente cronologico. Le strutture utilizzate sono:

1. `QHash<QString, model::Message>`⁴: HashMap che garantisce la ricerca con complessità $O(1)$ per l'estrazione rapida di un singolo messaggio tramite il suo ID.
2. `QQueue<QString>`⁵: mantiene l'ordine cronologico dei messaggi (FIFO).

Quando la cache raggiunge la capacità massima assegnata (`m_capacity`), l'elemento più vecchio viene rimosso (*evicted*) per fare spazio ai nuovi dati, tenendo traccia degli effettivi ultimi n messaggi.

Integrità dei dati e supporto alla resilienza

L'integrazione con `model::Message` permette alla cache di validare ogni inserimento tramite il metodo `msg.isValid()`. Questo assicura che solo dati strutturalmente corretti vengano memorizzati.

Il metodo di gestione `getMessagesSince(QString messageId)` analizza la coda cronologica e restituisce tutti i messaggi successivi ad un determinato ID. Questa funzione è vitale per gestire brevi interruzioni di rete, permettendo al frontend di ricostruire la cronologia mancante, senza dover consultare il server remoto, in quanto possibilmente già ricevuti e presenti nel buffer parziale.

4.4.2 Gestione della recovery

Il modulo `RecoveryManager` agisce come orchestratore per il riallineamento dello stato del client con il backend Appwrite. Il suo scopo è colmare lacune informative causate da instabilità della rete o avvii a freddo dell'applicazione.

⁴<https://doc.qt.io/qt-6/qhash.html>

⁵<https://doc.qt.io/qt-6/queue.html>

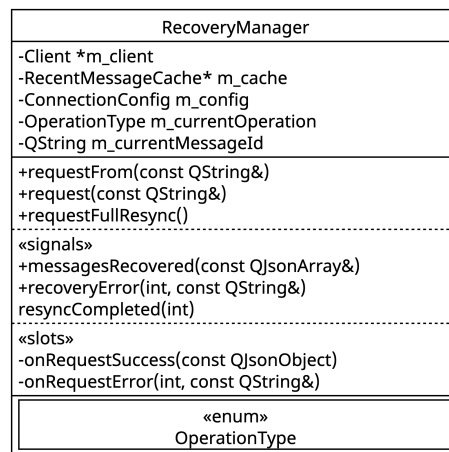


Figura 4.4: Struttura di RecoveryManager

Strategie di recupero e *Gap Detection*

Il modulo gestisce diverse modalità operative per garantire l'integrità dei dati; di seguito una breve descrizione:

1. **Recupero delta (`requestFrom`):** Identifica i messaggi mancanti rispetto all'ultimo ID noto. Se la cache non contiene l'intera sequenza, il manager effettua una query asincrona direttamente al backend Appwrite utilizzando i filtri temporali estratti dai metadati del messaggio.
2. **Sincronizzazione globale (`requestFullResync`):** Esegue una query per ottenere gli ultimi n messaggi dalla collezione, garantendo un punto di recovery certo, pensato per scenari di desincronizzazione critica.

Pipeline di trasformazione asincrona

Il manager funge da ponte tra il trasporto JSON ed il modello tipizzato. Quando l'SDK riceve dati dal server, il `RecoveryManager` esegue il parsing tramite `model::Message::fromJson()`, valida i messaggi e popola la cache, prima di emettere il segnale `messagesRecovered` verso il frontend.

4.4.3 Controllo del Throughput

La prevenzione del superamento delle quote API imposte dal server e la protezione dell'infrastruttura dai carichi eccessivi è di vitale importanza per qualsiasi servizio che voglia offrire un'esperienza di utilizzo decente. Per fare ciò è stato implementato un modulo di limitazione del traffico, basato sull'algoritmo *Token Bucket* [3].

Token Bucket [3]

Il sistema gestisce un "secchio" di token, che definisce la capacità massima di richieste simultanee, anche detta *burst size*. Un token viene consumato ad ogni richiesta e questi vengono rigenerati ad un ritmo costante, il quale è presente come stato `m_refillRate`. Se il numero di token disponibili è zero, la richiesta viene negata, quindi effettivamente messa in attesa finché, una volta che i token sono di nuovo disponibili, potrà riprovare; tutto ciò viene svolto tramite il metodo `allowRequest()`.

RateLimiter
-int m_availableTokens -QDateTime m_lastRefill -int m_maxTokens -int m_refillRate
+allowRequest() -> bool +availableTokens() -> int +maxTokens() -> int +refillRate() -> int +reset() -refillTokens()

Figura 4.5: Struttura di RateLimiter

Ottimizzazione delle risorse

L'implementazione adotta una strategia di refill passivo, detta *lazy refill*: invece di utilizzare timer attivi che graverebbero sulla CPU, il calcolo della rigenerazione avviene *on-demand* ogni volta che viene richiesto un accesso, misurando i millisecondi trascorsi dall'ultima operazione riuscita. Questo approccio garantisce un controllo preciso del traffico con un overhead computazionale trascurabile.

4.5 Gestione degli errori

Per gestire in modo uniforme i fallimenti all'interno del sistema, il progetto introduce un modello di errore centralizzato definito nel file `errorModel.h`. Questo modulo consente di rappresentare e propagare gli errori in maniera coerente tra frontend, proxy e backend.

L'obiettivo è evitare l'uso di codici di errore impliciti o messaggi non strutturati, fornendo invece una rappresentazione tipizzata e standardizzata delle condizioni di errore.

4.5.1 Classificazione degli errori

Gli errori sono classificati tramite l'enumerazione `ErrorCode`, che definisce un insieme finito di categorie:

Codice	Descrizione
None	assenza di errore
AuthError	errore di autenticazione
NetworkError	problemi di rete
ParseError	errori durante il parsing dei dati
ProtocolError	comportamento non conforme al protocollo
PermissionError	accesso negato
TimeoutError	timeout della richiesta
RateLimitExceeded	superamento dei limiti di richiesta
Unknown	errore non classificato

Tabella 4.1: Codici di errore

Questa classificazione permette di distinguere rapidamente la natura del problema e di gestirlo in modo appropriato nei diversi livelli dell'applicazione.

4.5.2 Struttura dell'errore

La struttura `AppcommError` rappresenta un errore completo all'interno del sistema. Essa include un codice di errore (`ErrorCode`), un messaggio descrittivo leggibile (`message`) ed un codice di stato HTTP opzionale (`httpStatus`), utile quando l'errore deriva da operazioni REST.

Questa struttura consente di trasportare sia informazioni tecniche sia informazioni utili per il debug o la visualizzazione lato utente.

4.5.3 Rilevazione e formattazione

Il metodo `hasError()` permette di verificare rapidamente se l'oggetto rappresenta un errore, controllando che il codice sia diverso da `ErrorCode::None`. Questo evita l'uso di controlli sparsi o condizioni implicite.

La funzione `errorCodeToString()` fornisce una rappresentazione testuale del codice di errore, utile per logging e debugging.

Il metodo `toString()` restituisce invece una stringa formattata che include il tipo di errore, il codice HTTP associato ed il messaggio descrittivo.

Ad esempio, un errore può essere rappresentato come:

```
Error[NetworkError] (500): Internal server error
```

4.5.4 Integrazione nel sistema

Il modello di errore è utilizzato trasversalmente nei diversi componenti del sistema; Mentre l'SDK segnala errori di rete o autenticazione, il parsing dei dati invece utilizza `ParseError` in caso di payload non validi, il proxy può generare errori di protocollo o permessi ed i moduli applicativi possono reagire in base al tipo di errore ricevuto.

L'uso di un modello centralizzato consente di evitare ambiguità e di uniformare la gestione degli errori in tutto il sistema, migliorando la robustezza e la leggibilità del codice.

4.5.5 Scelte progettuali

L'introduzione di un tipo dedicato per gli errori evita la propagazione di stringhe non strutturate o codici numerici non documentati. Questo approccio rende esplicito il significato degli errori e facilita l'estensione futura del sistema con nuove categorie o metadati aggiuntivi.

Inoltre, la separazione tra codice di errore, messaggio e stato HTTP permette di gestire in modo flessibile sia errori interni sia errori derivanti da servizi esterni.

4.6 Gestione dello stato del client

Per gestire il comportamento dinamico del client durante l'esecuzione, il progetto introduce un modello di stato definito nel file `state.h`. Questo modulo rappresenta lo stato runtime del client, includendo informazioni sulla connessione, sulla sessione e sul contesto di comunicazione attivo.

L'obiettivo è fornire una rappresentazione centralizzata dello stato del client, che possa essere condivisa tra i diversi componenti e utilizzata per sincronizzare logica applicativa e interfaccia utente.

4.6.1 Struttura dello stato

Lo stato del client è rappresentato dalla struttura `ClientState`, che contiene le informazioni necessarie per descrivere il contesto corrente della comunicazione. In particolare, essa include:

- **lastReceivedSequence**: numero di sequenza dell'ultimo messaggio ricevuto, utilizzato per garantire l'ordine dei messaggi e rilevare eventuali gap nella comunicazione, impostato di default a -1 per indicare che nessun messaggio è stato ancora ricevuto;
- **lastReceivedMessageId**: identificatore dell'ultimo messaggio ricevuto, utile per operazioni di recovery o deduplicazione;
- **activeTopicId**: identificatore del canale attualmente attivo, che definisce il contesto della comunicazione;
- **authenticated**: flag che indica se il client ha completato con successo il processo di autenticazione, impostato di default a false per indicare che l'utente non è autenticato.

Questa struttura rappresenta una fotografia dello stato corrente del client e viene aggiornata dinamicamente durante l'esecuzione.

4.6.2 Gestione dell'ordinamento e recovery

I campi `lastReceivedSequence` e `lastReceivedMessageId` sono fondamentali per garantire la correttezza del flusso dei messaggi. Il numero di sequenza permette di verificare l'ordine dei messaggi ricevuti, individuare eventuali messaggi mancanti (gap) e di supportare meccanismi di recovery basati su sequenze.

L'identificatore del messaggio, invece, può essere utilizzato per prevenire duplicazioni e per mantenere coerenza tra client e backend.

4.6.3 Contesto di comunicazione

Il campo `activeTopicId` definisce il contesto corrente in cui il client opera. Questo consente di filtrare i messaggi rilevanti e di limitare le operazioni al canale attivo.

Separare esplicitamente il canale attivo evita ambiguità e semplifica la gestione della comunicazione in presenza di più canali.

4.6.4 Stato di autenticazione

Il flag `authenticated` rappresenta lo stato di autenticazione del client. Questo valore è utilizzato per abilitare o disabilitare operazioni che richiedono una sessione valida.

In particolare, consente di impedire l'invio o la ricezione di messaggi quando il client non è autenticato, garantendo coerenza con le politiche di sicurezza del sistema.

4.6.5 Stato della connessione

Lo stato della connessione è rappresentato dall'enumerazione `ConnectionState`, che descrive il ciclo di vita della connessione tra client e backend.

Le possibili condizioni includono: (4.2)

Questa rappresentazione esplicita permette di gestire in modo chiaro il comportamento del sistema in funzione dello stato della rete.

Codice	Descrizione
Disconnected	assenza di errore
Connecting	errore di autenticazione
Connected	connessione attiva e funzionante;
Reconnecting	tentativo di ripristino dopo una disconnessione;
Disconnecting	chiusura della connessione in corso;
Failed	errore durante il tentativo di connessione.

Tabella 4.2: Stati di connessione

4.6.6 Integrazione con gli altri componenti

Il modello di stato è utilizzato da diversi moduli del sistema:

- il `MessageProcessor` utilizza `lastReceivedSequence` per verificare duplicati e gap;
- il `RecoveryManager` sfrutta le informazioni di stato per riallineare il client con il backend;
- i componenti di rete aggiornano lo stato della connessione;
- l'interfaccia utente può reagire ai cambiamenti di stato per abilitare o disabilitare funzionalità.

Questo approccio consente di mantenere una visione coerente e sincronizzata dello stato del sistema, migliorando la robustezza e la prevedibilità del comportamento del client.

4.6.7 Scelte progettuali

La separazione tra stato del client e logica operativa consente di ridurre l'accoppiamento tra i componenti e di semplificare la gestione della complessità.

Centralizzare lo stato in una struttura dedicata permette inoltre di facilitare il debugging, rendere esplicite le dipendenze tra componenti e supportare estensioni future del sistema senza modifiche invasive.

4.7 Elaborazione dei messaggi in ingresso

Per gestire in modo controllato i messaggi ricevuti dal backend, il client introduce il componente `MessageProcessor`. Questa classe ha il compito di analizzare i messaggi in ingresso prima che vengano propagati alla `façade Client` dell'applicazione.

L'obiettivo è garantire che tutti i messaggi validi, coerenti con il contesto corrente e non duplicati vengano accettati dal client. In questo modo, il componente contribuisce a mantenere la correttezza del flusso di comunicazione e a supportare i meccanismi di recovery (vedi 4.4.2).

4.7.1 Ruolo del componente

La classe `MessageProcessor` opera come filtro logico sui messaggi ricevuti. In particolare, essa verifica la validità strutturale del messaggio, la coerenza con il canale attualmente attivo, l'assenza di duplicati e la presenza di eventuali gap nella sequenza dei messaggi.

Solo i messaggi che superano tutti questi controlli vengono restituiti come validi al chiamante. La scelta di utilizzare un'unica collezione globale `messages` sul backend e di delegare il filtraggio per canale (in parte alle query API e in parte al processore locale) presenta un vantaggio architetturale fondamentale: evita la proliferazione dinamica di collezioni su Appwrite

(operazione complessa, lenta e soggetta a limiti di quota severi), garantendo uno schema del database fisso e scalabile. L'eventuale svantaggio è che il socket in tempo reale potrebbe ricevere pacchetti indesiderati se la stringa di sottoscrizione non è sufficientemente granulare; tuttavia questo viene compensato efficacemente proprio dal controllo rigoroso implementato a questo livello del client, che isola l'interfaccia utente da flussi di dati estranei.

4.7.2 Dipendenza dallo stato del client

Il processore mantiene un riferimento a un oggetto `ClientState`, da cui ricava il contesto runtime necessario per prendere decisioni sui messaggi ricevuti. In particolare, vengono utilizzati:

- `activeTopicId`, per verificare che il messaggio appartenga al canale attualmente selezionato;
- `lastReceivedSequence`, per controllare l'ordine dei messaggi e individuare duplicati o gap.

Questo rende `MessageProcessor` strettamente integrato con il modello di stato del client, senza però incorporare direttamente la logica di connessione o di recovery.

4.7.3 Metodo `processIncoming()`

Il metodo principale della classe è `processIncoming()`, che riceve un oggetto `model::Message` e restituisce un valore di tipo `std::optional<model::Message>`.

Questa scelta progettuale permette di esprimere in modo esplicito due possibili esiti:

- presenza di un messaggio valido, che può essere elaborato dai moduli successivi;
- assenza di risultato, nel caso in cui il messaggio venga scartato.

L'elaborazione avviene verificando la validità del messaggio tramite `message.isValid()`, successivamente controllando il canale attivo, scartando i messaggi che non appartengono al contesto corrente, poi rilevando i duplicati tramite `isDuplicate()`, dopo di che rilevando eventuali gap tramite `hasGap()` e infine aggiornando lo stato del client con l'ultima sequenza ricevuta.

Se uno dei controlli fallisce, il metodo restituisce `std::nullopt`. In caso contrario, restituisce il messaggio originale.

4.7.4 Deduplicazione

La deduplicazione è affidata al metodo `isDuplicate()`, che confronta il numero di sequenza del messaggio in ingresso con il valore di `lastReceivedSequence` memorizzato nello stato del client.

Un messaggio viene considerato duplicato se la sua sequenza è minore o uguale all'ultima sequenza già accettata. Questa scelta impedisce che messaggi ricevuti più volte vengano propagati nuovamente all'applicazione.

4.7.5 Rilevazione di gap

Il metodo `hasGap()` verifica se esiste un salto nella sequenza dei messaggi ricevuti. Se non è ancora stato ricevuto alcun messaggio, cioè se `lastReceivedSequence` è inizializzato a `-1`, il metodo restituisce `false`.

Negli altri casi, viene rilevato un gap quando la sequenza del nuovo messaggio è maggiore di `lastReceivedSequence + 1`. Questo segnala la possibile perdita di uno o più messaggi e costituisce il punto di aggancio per i meccanismi di recovery.

4.7.6 Aggiornamento dello stato

Quando un messaggio viene accettato, il processore aggiorna il campo `lastReceivedSequence` nello stato del client. In questo modo, i messaggi successivi possono essere confrontati rispetto all'ultima sequenza valida già elaborata.

Questo aggiornamento centralizzato garantisce coerenza tra il flusso dei messaggi ricevuti e lo stato runtime mantenuto dal client.

4.7.7 Reset del contesto

Il metodo `reset()` consente di reinizializzare il processore rispetto a un nuovo canale attivo. In particolare, esso:

- aggiorna il valore di `activeTopicId`;
- reimposta `lastReceivedSequence` a `-1`.

Questa operazione è necessaria quando il client cambia canale, poiché la sequenza dei messaggi deve essere considerata nuovamente a partire da uno stato iniziale.

4.7.8 Scelte progettuali

La classe `MessageProcessor` separa la logica di validazione del flusso dei messaggi dalla logica di trasporto e dalla logica applicativa. Questo riduce l'accoppiamento tra i componenti e rende più semplice testare in modo isolato i casi di duplicazione, disallineamento e filtro per canale.

L'uso combinato di `ClientState`, `model::Message` e `std::optional` rende inoltre il comportamento del componente esplicito, prevedibile e facilmente estendibile con future strategie di recovery.

4.8 Comunicazione in tempo reale

Per supportare la ricezione immediata di eventi (ovvero le notifiche generate istantaneamente dal backend Appwrite quando un nuovo messaggio viene salvato nel database), il sistema integra un modulo dedicato alla comunicazione WebSocket. La classe `Realtime` fornisce un'interfaccia semplificata per sottoscrivere i canali di eventi e reagire senza latenze artificiali.

4.8.1 Architettura del modulo Realtime

Il modulo `Realtime` si basa su `QWebSocket`, la libreria Qt per la gestione di connessioni WebSocket. Questa scelta permette di mantenere la compatibilità con `WebAssembly` e di integrarsi naturalmente con il sistema di segnali e slot già utilizzato nel resto della libreria.

La classe si occupa di stabilire una connessione WebSocket con l'endpoint Appwrite, di inviare la richiesta di sottoscrizione ai canali desiderati e di interpretare i messaggi JSON ricevuti dal server.

Realtime
-QWebSocket *m_webSocket -ConnectionConfig m_config;
+connect(const QStringList&) +disconnect()
«signals» +connected() +disconnected() +eventReceived(const QJsonObject&) +errorOccurred(const QString&)
«slots» -onConnected() -onDisconnected() -onTextMessageReceived(const QString&) -onErrorOccurred(SocketError)

Figura 4.6: Struttura di Realtime

4.8.2 Sottoscrizione ai canali

Appwrite organizza gli eventi secondo un modello gerarchico. *Nota Terminologica:* Per evitare ambiguità, è importante distinguere i "Canali Logici" (le stanze di chat identificate dal `topicId` nel nostro dominio) dai "Canali Realtime" (o *Topics*) utilizzati dall'API di Appwrite per il multiplexing degli eventi.

Il formato dei canali realtime di Appwrite segue la convenzione:

```
databases.{databaseId}.collections.{collectionId}.documents.{documentId}
```

È possibile sottoscrivere a più canali contemporaneamente, permettendo al client di ricevere notifiche da diverse risorse con una singola connessione WebSocket.

Il metodo `connect()` accetta una lista di canali e costruisce automaticamente l'URL WebSocket a partire dalla configurazione fornita, sostituendo il protocollo HTTP con WS (o HTTPS con WSS per connessioni sicure).

4.8.3 Gestione degli eventi

Quando il backend Appwrite rileva un cambiamento in una risorsa sottoscritta, invia un messaggio JSON attraverso il WebSocket. Il modulo Realtime intercetta questi messaggi tramite lo slot `onTextMessageReceived()`, esegue il parsing JSON e propaga l'evento tramite il segnale `eventReceived()`.

La struttura degli eventi include informazioni sul tipo di cambiamento (creazione, modifica, cancellazione), i canali interessati, il timestamp dell'evento e il payload contenente i dati aggiornati della risorsa.

Questo approccio permette ai moduli superiori di reagire immediatamente ai cambiamenti senza dover implementare meccanismi di polling, riducendo la latenza e il carico sul server.

4.8.4 Gestione della connessione

Il modulo emette segnali distinti per ogni fase del ciclo di vita della connessione. Il segnale `connected()` viene emesso quando la connessione WebSocket è stabilita con successo, mentre `disconnected()` segnala la chiusura della connessione.

In caso di errore durante la connessione o la comunicazione, viene emesso il segnale `errorOccurred()`, contenente una descrizione dell'errore utile per il debug o per implementare logiche di riconnessione.

4.8.5 Integrazione con il sistema

Il modulo `Realtime` opera in modo indipendente dal resto della libreria, comunicando esclusivamente attraverso segnali Qt. Questo disaccoppiamento permette di sostituire facilmente il meccanismo di comunicazione in tempo reale o di integrare strategie di fallback basate su polling.

Nel contesto del client, il modulo viene utilizzato per ricevere notifiche immediate di nuovi messaggi nei canali attivi, evitando la necessità di interrogare periodicamente il server per verificare la presenza di aggiornamenti.

4.9 Pulizia automatica dei dati

Per garantire l'igiene del database e rispettare le politiche di retention dei dati, il sistema server integra un modulo di garbage collection automatizzato. La classe `GarbageCollector` si occupa di eliminare messaggi obsoleti in base a politiche di TTL configurabili.

4.9.1 Motivazione e contesto

Nel corso del tempo, il database di messaggi può accumulare un volume significativo di dati storici che non sono più rilevanti per le operazioni correnti. Mantenere indefinitamente tutti i messaggi comporta costi di storage crescenti e può degradare le prestazioni delle query.

Il `GarbageCollector` risolve questo problema implementando una strategia di eliminazione basata su tempo, rimuovendo automaticamente i messaggi che superano una soglia di età definita dalla policy di persistenza.

GarbageCollector
-Server* m_server -ConnectionConfig m_config -QStringList m_docsToDelete -int m_deletedCount
+runCleanup(const PersistencePolicy)
«signals»
+cleanupComplete(int)
+cleanupError(const QString&)
«slots»
-onDocumentsListed(const QJsonObject&)
-onDocumentDeleted(const QJsonObject&)
-onError(int code, const QString&)

Figura 4.7: Struttura di `GarbageCollector`

4.9.2 Policy di persistenza

Il modulo opera sulla base di una `PersistencePolicy`, che specifica il numero di giorni oltre i quali un messaggio viene considerato obsoleto. Se il campo `messageTTL` della policy è impostato a zero o a un valore negativo, il garbage collector non esegue alcuna operazione.

Questa flessibilità permette di adattare il comportamento del sistema a diverse esigenze operative, da ambienti di sviluppo dove si desidera conservare tutti i dati, a sistemi di produzione con vincoli di storage o requisiti di compliance.

4.9.3 Algoritmo di pulizia

L'operazione di pulizia procede in modo iterativo per evitare di sovraccaricare il server con richieste massive. Il processo si articola nelle seguenti fasi:

1. Calcolo della data di cutoff sottraendo il TTL dal timestamp corrente;
2. Query al database per recuperare fino a 100 messaggi antecedenti la data di cutoff;
3. Accodamento degli identificatori dei documenti da eliminare;
4. Eliminazione sequenziale di ogni documento tramite l'SDK server;
5. Ripetizione del processo finché non vengono più trovati messaggi obsoleti.

Questo approccio batch garantisce che anche in presenza di volumi elevati di dati da eliminare, il sistema possa completare l'operazione senza timeout o esaurimento delle risorse.

4.9.4 Gestione asincrona

Il `GarbageCollector` opera in modo completamente asincrono, utilizzando segnali Qt per comunicare l'avanzamento e il completamento dell'operazione. Durante l'esecuzione, ogni eliminazione incrementa un contatore interno che tiene traccia del numero totale di messaggi rimossi.

Al termine dell'operazione, viene emesso il segnale `cleanupComplete()` con il conteggio finale dei messaggi eliminati. In caso di errore durante la query o l'eliminazione, viene invece emesso `cleanupError()` con una descrizione dell'errore.

4.9.5 Privilegi e sicurezza

Il garbage collector richiede privilegi amministrativi per eliminare documenti dal database. Per questo motivo, utilizza l'istanza `appwritesdk::Server` che si autentica tramite API key, garantendo che solo il backend possa invocare operazioni di pulizia.

Questa separazione dei ruoli protegge l'integrità dei dati, impedendo ai client di eliminare arbitrariamente messaggi dal database.

4.9.6 Integrazione nel ciclo di vita del server

Il garbage collector può essere invocato periodicamente dal server tramite timer o in risposta a eventi specifici, come il superamento di soglie di storage o in finestre di manutenzione programmate.

La natura asincrona del modulo permette di eseguire la pulizia senza bloccare altre operazioni del server, mantenendo la reattività del sistema anche durante la rimozione di grandi quantità di dati.

4.10 API per Server

Il modulo `AppcommServer` è pensato per operare come orchestratore centrale per le operazioni amministrative del sistema. L'interfaccia pubblica è stata mantenuta minimale per nascondere la complessità degli stati interni, delegata ad una classe `Private`, secondo il pattern PIMPL⁶.

Il flusso di operazioni del server si divide in tre fasi distinte, le quali sono coordinate da segnali e slot per gestire la natura asincrona delle risposte di rete.

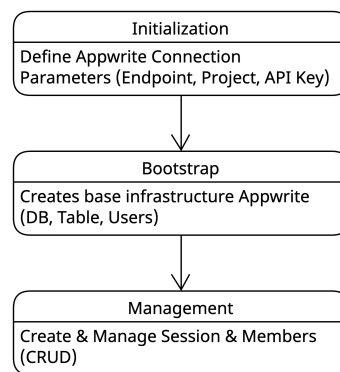


Figura 4.8: Lifecycle di `AppcommServer`

Ogni metodo pubblico valida internamente lo stato dell'istanza; ad esempio le operazioni sui canali emettono un errore immediato nel caso il server non abbia completato con successo la fase di bootstrap.

4.10.1 Logica di Bootstrap

L'inizializzazione dell'infrastruttura rappresenta la fase più critica del modulo `AppcommServer`. A differenza di un database standard, il bootstrap per Appwrite richiede la creazione sequenziale di database, collezioni e attributi. Questa è quindi implementata come una catena di chiamate asincrone sequenziali, la quale, dato che Appwrite richiede che ogni risorsa sia dapprima creata per potervi aggiungere delle proprietà, viene gestita con una serie di passaggi obbligati:

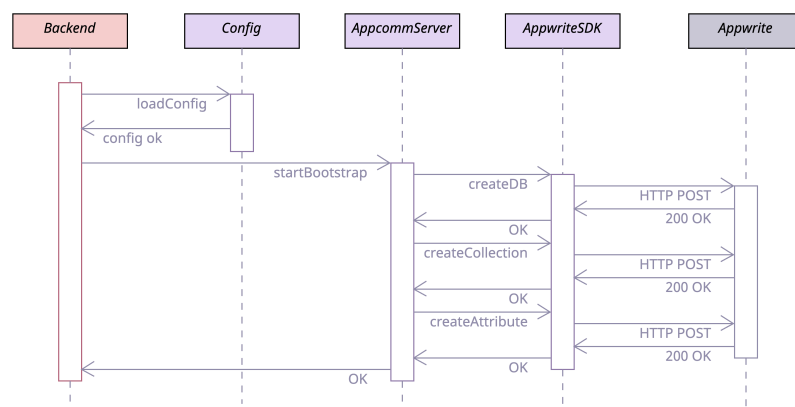


Figura 4.9: UML Sequence, processo di Bootstrap

⁶<https://en.cppreference.com/w/cpp/language/pimpl.html>

1. Viene inviata la richiesta iniziale per la creazione del database⁷; nel caso il database già esista (codice errore 409), il sistema lo rileva e prosegue automaticamente;
2. Vengono poi create le collezioni per i messaggi, i membri ed i canali. (Sul backend, la nozione di "canale" viene tradotta fisicamente in un contenitore di documenti all'interno della collezione `channels`);
3. Il server richiede la creazione della struttura (gli `Attributes`) all'interno di queste collezioni, uno alla volta, attendendo la conferma da parte del server;
4. Per ottimizzare la performance della recovery, vengono creati anche degli indici (`Indexes`) specifici, come ad esempio quelli sul timestamp.

Il segnale `initialized()` viene emesso solo al termine di questa sequenza, garantendo l'integrità dell'infrastruttura e la prontezza ad accogliere i dati, senza errori di schema.

4.10.2 Gestione dei canali e della membership

Questo componente funge anche da autorità centrale per la gestione degli spazi di comunicazione. Questa responsabilità è suddivisa tra la creazione fisica di risorse e la gestione di relazioni tra utenti.

Creazione e permessi

All'invocazione del metodo `createTopic()`, il server non si limita solamente alla creazione di un documento (che andrà a popolare la collezione `channels` con i metadati di base come `topicId`, `name` e `createdAt`), bensì definisce delle regole d'accesso affinché i client possano interagire con il canale in modo sicuro. Questi permessi vengono configurati in modalità "broadcast", permettendo la lettura a tutti i membri autorizzati.

Gestione dei membri

Attraverso il modulo `MembershipService`, il server è in grado di gestire l'aggiunta e la rimozione degli utenti dai canali. L'operazione `addMember()` associa un utente, tramite il suo identificativo, ad un canale, sempre utilizzando l'ID di quest'ultimo, all'interno di una collezione dedicata. Ciò permette al sistema di filtrare correttamente i messaggi in tempo reale per gli utenti attualmente connessi.

Pulizia e manutenzione

Il server implementa altresì logiche di eliminazione a cascata: la rimozione di un canale comporta anche l'epurazione di tutti i dati associati, quindi i messaggi e i riferimenti di membership. Ciò garantisce l'integrità del database ed il rispetto delle politiche di persistenza.

4.10.3 Provisioning

Il bootstrap, quindi il provisioning delle strutture dati necessarie, rappresenta il cuore del Proof of Concept, validando la capacità della libreria di gestire sequenze complesse di inizializzazione.

Viene eseguita dapprima una verifica dello stato remoto, dove il sistema verifica la presenza dell'infrastruttura, seguita, solamente se necessario e solo per i componenti necessari, la creazione sequenziale ed infine viene notificata la prontezza, sbloccando le funzionalità operative del `BackendService`.

⁷Per istanze non-premium il limite è di massimo un database

4.11 API per Client

4.11.1 Descrizione generale

La classe `AppcommClient` rappresenta la *façade* di alto livello del lato client del sistema `apcomm`. Essa fornisce un'interfaccia pubblica semplificata per i layer di presentazione (ad esempio QML o Qt Widgets, che nel nostro caso specifico è il contenuto di del frontend), nascondendo la complessità dei componenti sottostanti tramite una classe `Private` secondo il pattern PIMPL⁸, come fatto per l'API dedicata al Server.

La classe si occupa di coordinare l'autenticazione dell'utente, la gestione del canale attivo, il caricamento iniziale dei messaggi dal database, la ricezione dei messaggi in tempo reale, la gestione del recupero di messaggi mancanti, e l'esposizione dello stato di connessione alla UI.

L'implementazione segue un approccio a composizione, delegando responsabilità specifiche a componenti interni specializzati.

4.11.2 Componenti interni

`AppcommClient` utilizza i seguenti componenti:

- **Client (Appwrite SDK)**: gestione delle richieste HTTP verso il backend
- **Realtime**: gestione della connessione WebSocket e ricezione eventi
- **MessageProcessor**: validazione, deduplicazione e rilevamento dei gap nei messaggi
- **RecoveryManager**: recupero dei messaggi mancanti
- **RecentMessageCache**: cache locale dei messaggi recenti
- **RateLimiter**: limitazione delle richieste in uscita

4.11.3 Flusso operativo

Il comportamento del client segue il seguente flusso (4.10):

1. Creazione dell'istanza con una configurazione valida
2. Autenticazione dell'utente (guest o email/password)
3. Recupero della membership dell'utente loggato con email dal database
4. Identificazione del canale associato all'utente
5. Caricamento dei messaggi esistenti ordinati per `sequenceNumber`
6. Connessione al sistema realtime per ricevere nuovi eventi
7. Invio e ricezione dei messaggi

4.11.4 Gestione dello stato

La classe mantiene due principali tipi di stato:

ConnectionState: rappresenta lo stato della connessione realtime

⁸<https://en.cppreference.com/w/cpp/language/pimpl.html>

```
enum class ConnectionState {
    Disconnected = 0, ///< No active connection
    Disconnecting,    ///< Connection is being closed
    Connected,        ///< Connection is established and active
    Connecting,       ///< Connection attempt in progress
    Reconnecting,     ///< Attempting to restore a lost connection
    Failed            ///< Connection attempt failed
};
```

ClientState: contiene informazioni operative, tra cui:

```
struct ClientState {
    qint64 lastReceivedSequence = -1; ///< Sequence last message received
    QString lastReceivedMessageId;    ///< ID of the last message received
    QString activeTopicId;            ///< Id of the currently active topic channel
    bool authenticated = false;       ///< Whether client is authenticated
};
```

4.11.5 Gestione dell'autenticazione

Il client supporta due modalità di autenticazione, la sessione anonima oppure la sessione autenticata con email e password.

Per quanto riguarda la sessione anonima, di default da Appwrite i guest non hanno i permessi di read o write, quindi non potranno interagire con il client. Invece chi esegue il login direttamente con l'email e la password, avrà già il canale associato a lui.

Dopo una risposta valida dal server, vengono salvate le informazioni di sessione, lo stato di autenticazione viene aggiornato e viene avviato automaticamente il caricamento della membership.

4.11.6 Gestione del canale

Il client mantiene un singolo canale attivo per volta.

Quando viene chiamato `joinTopic()` il canale viene validato e salvato, lo stato del `MessageProcessor` viene resettato, viene emesso il segnale `joinedTopic` ed infine viene avviato il caricamento dei messaggi dal database.

Quando viene chiamato `leaveTopic()` il canale attivo viene rimosso, viene emesso il segnale `leftTopic` e la connessione realtime viene chiusa se attiva.

4.11.7 Caricamento dei messaggi

I messaggi vengono caricati dal database tramite query filtrate per `topicId` e ordinate per `sequenceNumber` crescente.

Ogni messaggio recuperato viene dapprima convertito in oggetto `Message`, viene poi processato dal `MessageProcessor`, se valido, viene inserito nella cache, ed infine viene emesso tramite il segnale `messageReceived`.

Se il client non è connesso al termine del caricamento, viene avviata automaticamente la connessione realtime.

4.11.8 Ricezione realtime

Gli eventi ricevuti tramite `WebSocket` vengono normalizzati per estrarre il payload reale, convertiti in oggetti `Message`, filtrati tramite il `MessageProcessor`, salvati nella cache e poi emessi verso la UI.

4.11.9 Invio dei messaggi

L'invio di un messaggio richiede i seguenti prerequisiti; un utente autenticato, un canale attivo, disponibilità del rate limiter, e configurazione della collection di input.

Il messaggio viene:

- arricchito con identificatore univoco e timestamp
- convertito in formato JSON
- salvato nella collection di messaggi in ingresso

Il backend sarà responsabile della trasformazione nel messaggio finale.

4.11.10 Gestione del recupero

In presenza di gap nei `sequenceNumber`, il `RecoveryManager` viene attivato automaticamente dalla classe `AppcommClient`. Quando il `MessageProcessor` restituisce esito negativo alla verifica `hasGap()`, il client invoca il manager passando l'ID dell'ultimo messaggio valido ricevuto, così da richiedere in background il blocco di messaggi mancante e ricucire la cronologia.

I messaggi recuperati seguono lo stesso flusso:

Validazione → deduplicazione → inserimento in cache → emissione verso la UI.

4.11.11 Gestione degli errori

Gli errori provenienti da richieste HTTP, oppure connessione realtime o anche validazioni locali vengono propagati tramite il segnale:

```
errorOccurred(QString)
```

4.11.12 Limitazioni attuali

L'implementazione corrente presenta alcune semplificazioni progettuali:

- è supportata una sola richiesta logica alla volta (tramite `PendingRequest`)
- si assume una singola membership valida per utente
- il filtro per canale viene applicato lato client sui messaggi ricevuti

Queste scelte semplificano il design, mantenendo il comportamento prevedibile e coerente con i requisiti del progetto.

4.12 Conclusione

Il capitolo ha mostrato che l'implementazione non è una sequenza di componenti isolati, ma un sistema coerente in cui il dominio, trasporto, stato, errori e resilienza collaborano secondo responsabilità chiaramente separate. Questa decomposizione riduce l'accoppiamento, facilita il testing e permette di intervenire su singole parti senza introdurre regressioni trasversali.

Dal punto di vista operativo la combinazione tra SDK asincrono, processamento dei messaggi, recovery, realtime e policy di pulizia automatica realizza un'infrastruttura robusta rispetto a disconnessioni, duplicazioni e carichi variabili. Parallelamente il layer server espone primitive amministrative sufficienti per governare il provisioning, canali e membership, mantenendo centralizzate le operazioni privilegiate.

Il frontend completa questa struttura fornendo un punto di accesso concreto alle funzionalità del client. Attraverso il controller Qt e le schermate QML, lo stato interno di `AppcommClient` viene reso osservabile dall'utente: autenticazione, topic attivo, messaggi ricevuti, errori e stato della connessione realtime vengono propagati all'interfaccia senza spostare nella vista la logica di comunicazione. In questo modo, il frontend rimane un layer di presentazione leggero, ma sufficiente a dimostrare l'integrazione della libreria in un'applicazione reale.

Nel complesso, le scelte implementative traducono i principi architetturali in un artefatto software (ri)utilizzabile. Questo costituisce la base su cui il capitolo successivo presenta il Proof of Concept, ossia la validazione pratica del modello attraverso un flusso end-to-end.

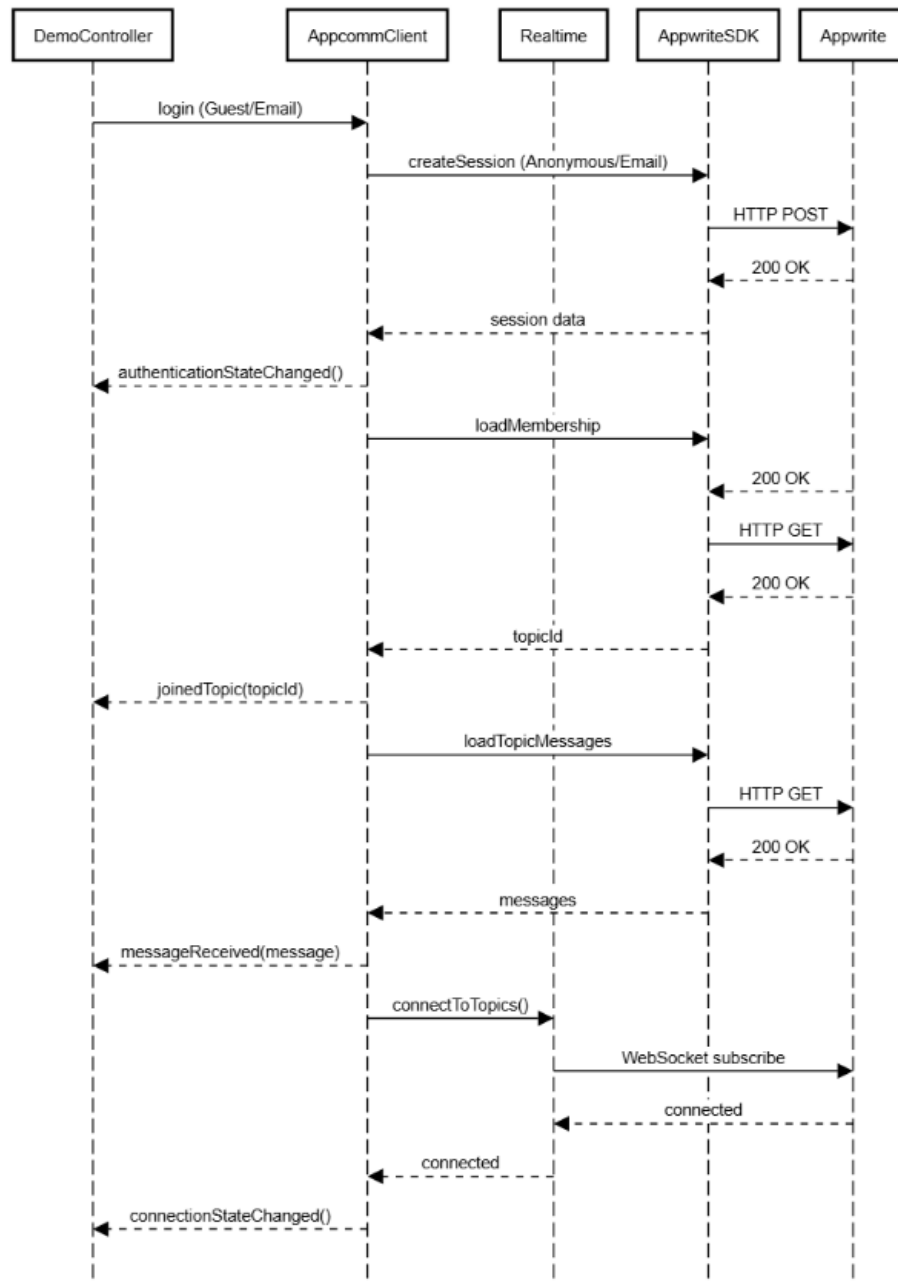


Figura 4.10: Sequence Diagram UML inizializzazione e sincronizzazione Client.

Capitolo 5

Proof of Concept

Questo capitolo descrive l'implementazione del software di test e dimostrazione, utilizzato per validare l'architettura ed il funzionamento della libreria `appcomm`, come per fornire un esempio per una futura integrazione in qualsiasi altro sistema.

Il *Proof of Concept* è un'applicazione standalone, che simula il comportamento di un server di provisioning e gestione, con un client che invia e riceve messaggi attraverso l'infrastruttura.

La finalità del PoC non è coprire tutti i casi di utilizzo in produzione, ma bensì di dimostrare in modo riproducibile che i meccanismi fondamentali progettati nei capitoli precedenti funzionino in un contesto reale; bootstrap automatico dell'ambiente Appwrite, gestione amministrativa delle entità, pubblicazione e ricezione dei messaggi, integrazione con il canale realtime e comportamento di risposta del backend in modalità "echo".

Il PoC include inoltre un frontend dimostrativo realizzato con Qt/QML, pensato per validare l'integrazione della libreria `appcomm` dal punto di vista di un'applicazione client. L'interfaccia grafica dialoga con la libreria tramite un controller Qt, che espone a QML le funzionalità essenziali di `AppcommClient`: autenticazione, invio dei messaggi, visualizzazione dello stato di connessione e logout. Il frontend non ha lo scopo di coprire tutte le funzionalità di un'applicazione finale, ma di fornire un'interfaccia sufficiente a verificare il funzionamento end-to-end del sistema.

In questa prospettiva, il PoC agisce come banco di prova; collega la teoria architetturale alla pratica esecutiva e rende osservabili i punti critici del sistema, come la coerenza della configurazione, l'ordine dei messaggi e la correttezza del provisioning iniziale.

5.1 Backend

Il backend di un sistema che comunica tramite la libreria appcomm non ha tante responsabilità quante ne avrebbe in un'architettura classica; infatti non ha nessun bisogno di gestire un'istanza di database, né di un sistema di autenticazione o di altri complessi componenti che si trovano tradizionalmente nei backend. Questi compiti sono infatti delegati al componente *BaaS* (3.4) hostato su Appwrite e controllato dalla libreria.

Le responsabilità del backend sono dapprima il provisioning della struttura necessaria per contenere i dati, su Appwrite e la fornitura di un'interfaccia per gestione amministrativa dei dati e degli utenti sulla piattaforma.

5.1.1 Requisiti di validazione

ID	Descrizione	Note
R-13	<i>Backend</i> permette di richiedere i messaggi di un canale	-
R-14	<i>Backend</i> fornisce un servizio di "echo" subscribe + republish	-
R-15	<i>Backend</i> gestisce la configurazione dell'istanza di Appwrite	-
R-16	<i>Backend</i> gestisce le sessioni aperte	-
R-17	<i>Backend</i> gestisce il lifecycle dei canali	-
R-18	<i>Backend</i> gestisce la membership di utenti nei canali	-
R-19	<i>Backend</i> gestisce la creazione e cancellamento di utenti	-

Tabella 5.1: Requisiti di validazione Backend

5.1.2 Architettura del sistema

Il PoC è strutturato per separare la configurazione statica dalla logica operativa, facilitando il test di diversi scenari di deployment. È infatti presente un modulo *BackendConfig*, il quale è dedicato alla gestione dei parametri di ambiente e delle credenziali SDK. Carica in modo sicuro l'endpoint, il *project-id* e l'*api-key* necessari per l'autenticazione privilegiata. È altresì presente il modulo *BackendService*, il quale agisce come il controller principale dell'applicazione, interfacciandosi tra la logica di business ed il server Appwrite, usando come tramite la libreria. È infine presente una classe che si occupa di garantire che l'ambiente remoto sia configurato correttamente prima di ogni operazione.

5.1.3 Assegnazione del numero di sequenza

Il backend risulta utile anche per risolvere un problema architetturale del sistema; infatti la gestione del numero di sequenza non può essere svolta dal client in un contesto dove i messaggi si trovano nella stessa tabella per tutti i canali, in quanto non è capace di sapere quale sarebbe il prossimo da assegnare.

Questo compito viene coperto dal server il quale, tramite un sistema di *Inbox/Outbox*, assegna e sposta i messaggi verso chi deve leggerli: sono presenti due collection, all'interno di Appwrite, una per i messaggi in entrata, ai quali non è ancora assegnato il sequence number (outbox), una per i messaggi per i quali la sequenza è presente e sono stati inoltrati (inbox).

Il backend server è in ascolto sulla collezione *pendingmessages* (outbox) e, alla ricezione di un nuovo messaggio, questo viene copiato nella collezione *messages* (inbox) e gli viene assegnato il corretto sequence number (5.1).

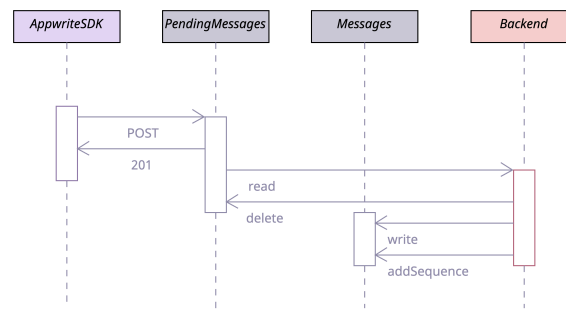


Figura 5.1: Sequence Diagram del sistema Inbox/Outbox

5.2 Frontend

Il frontend del PoC rappresenta il punto di ingresso lato utente per interagire con la libreria appcomm. A differenza di un'applicazione completa, il suo scopo non è fornire un'interfaccia definitiva o coprire tutti gli scenari di utilizzo, ma dimostrare in modo immediato e osservabile il comportamento del client durante le operazioni fondamentali: autenticazione, caricamento della cronologia dei messaggi, invio dei messaggi, ricezione realtime e logout.

L'interfaccia è stata realizzata con Qt/QML e si appoggia a un controller Qt che espone alla vista le funzionalità principali del client. La logica di comunicazione rimane incapsulata all'interno di `AppcommClient`, mentre il frontend si occupa di rappresentare lo stato dell'applicazione e di inoltrare le azioni dell'utente alla libreria tramite un controller.

5.2.1 Requisiti di validazione

ID	Descrizione
R-20	Il frontend permette l'accesso tramite sessione guest
R-21	Il frontend permette l'accesso tramite credenziali email
R-22	Il frontend visualizza lo stato della connessione realtime
R-23	Il frontend mostra i messaggi caricati dal topic assegnato
R-24	Il frontend permette l'invio di nuovi messaggi
R-25	Il frontend aggiorna la chat alla ricezione di nuovi messaggi
R-26	Il frontend gestisce il caso di accesso guest non autorizzato
R-27	Il frontend permette il logout e il ritorno alla schermata iniziale

Tabella 5.2: Requisiti di validazione Frontend

Nota: per quanto riguarda R-20, l'accesso è permesso tramite un bottone sulla schermata iniziale di login, però il guest non potrà accedere alla chat e gli apparirà invece una schermata di accesso negato perché ad oggi non esiste un'implementazione che gestisce il caso in cui il guest abbia i permessi e di default i permessi per il guest sono disabilitati.

5.2.2 Architettura del sistema

Il frontend è composto da schermate QML e da un controller C++ che funge da livello di adattamento tra l'interfaccia grafica e la libreria appcomm. Le schermate QML gestiscono esclusivamente la presentazione e le interazioni dell'utente, mentre il controller espone proprietà e metodi invocabili da QML, come login, invio messaggio e logout.

Il componente centrale lato client è `AppcommClient`. Dopo una richiesta di autenticazione, esso crea la sessione `Appwrite`, recupera la membership associata all'utente, determina il `topicId` assegnato e carica gli ultimi 16 messaggi disponibili. Al termine della sincronizzazione iniziale, il client apre una sottoscrizione realtime alla collection dei messaggi, così da ricevere automaticamente gli aggiornamenti prodotti dal backend.

Il controller riceve gli eventi emessi da `AppcommClient` tramite signal Qt e aggiorna lo stato esposto all'interfaccia grafica. In particolare, lo stato di autenticazione, l'identificativo dell'utente autenticato, lo stato della connessione, il topic corrente, gli errori e la lista dei messaggi sono mantenuti nel controller e notificati alla vista QML. Questo permette di mantenere separata la logica di comunicazione dalla logica di visualizzazione.

5.2.3 Flusso operativo del client

Il flusso operativo principale inizia dalla schermata di login, dalla quale l'utente può scegliere l'accesso guest oppure l'autenticazione tramite email. In entrambi i casi il controller inoltra la richiesta ad `AppcommClient`, che delega la creazione della sessione ad `Appwrite` tramite la nostra SDK.

Una volta ricevuta la sessione, il client salva in memoria le informazioni essenziali dell'utente, come l'identificativo della sessione, l'identificativo dell'utente e il tipo di autenticazione. Successivamente interroga la collection delle membership per individuare il topic associato all'utente. Se la membership è valida, il topic viene impostato come topic attivo e il frontend può accedere alla schermata di chat. Se invece non viene trovata alcuna membership associata all'utente, `AppcommClient` emette l'errore `No membership found for current user.`; nel caso di accesso guest, il controller interpreta questo errore come accesso non autorizzato e mostra la schermata dedicata. Se il documento di membership è presente ma non contiene dati validi, viene invece emesso l'errore `Invalid membership document.`

Prima di aprire la connessione realtime, il client carica una porzione iniziale della cronologia dei messaggi. Nel PoC vengono recuperati gli ultimi 16 messaggi disponibili in base al `sequenceNumber` che viene assegnato dal backend al momento della pubblicazione del messaggio definitivo nella collection dei messaggi; i documenti ricevuti vengono poi riordinati localmente in ordine crescente di sequenza prima di essere mostrati nella chat. Questa scelta permette di presentare all'utente il contesto più recente della conversazione senza dover scaricare l'intero storico.

Dopo il caricamento iniziale, `AppcommClient` stabilisce la connessione realtime verso `Appwrite`. Da questo momento, ogni nuovo messaggio pubblicato nella collection viene ricevuto come evento realtime, elaborato dal client e propagato al controller tramite il signal di ricezione del messaggio. Il modello QML viene quindi aggiornato e la chat mostra automaticamente il nuovo contenuto.

5.3 Porting WebAssembly

È stata prevista una variante sperimentale del frontend compilabile per WebAssembly, con l'obiettivo di rendere la demo accessibile direttamente dal browser. Questa versione riutilizza la stessa interfaccia Qt/QML e la stessa logica di comunicazione della versione desktop, ma viene compilata per l'esecuzione in ambiente browser tramite Qt for WebAssembly [4].

Poiché il kit Qt for WebAssembly utilizzato include il modulo `Qt WebSockets`, anche la variante browser può utilizzare la sottoscrizione realtime verso `Appwrite`. In questo modo il PoC valida non solo il porting dell'interfaccia nel browser, ma anche il mantenimento del modello di comunicazione realtime già adottato nella versione desktop.

Capitolo 6

Test e validazione

Questo capitolo presenta la validazione del progetto in modo analitico, con l'obiettivo di dimostrare non solo che i componenti funzionino, ma anche *perché* ciascun gruppo di test è rilevante rispetto all'architettura proposta. La libreria `appcomm` è composta da moduli con responsabilità diverse (modelli, logica client, orchestrazione server, integrazione Appwrite, gestione realtime e recovery): per questo motivo, la strategia di test è stata organizzata in suite tematiche, ognuna focalizzata su un comportamento osservabile e verificabile in modo ripetibile.

Il framework usato è `QTest`, con orchestrazione `cctest`. In ambiente locale ed in integrazione continua viene eseguita la stessa pipeline di istruzioni; viene dapprima configurato `CMake` con `BUILD_TESTING=ON`, viene avviata la build dei target di test e al completamento vengono eseguiti tutti i binari di test `test_*`. Questa scelta garantisce coerenza tra ambiente di sviluppo e l'ambiente CI, riducendo il rischio di discrepanze tra risultati ottenuti in laboratorio e in repository.

L'analisi seguente è costruita per moduli: per ogni suite vengono esplicitati intenti tecnici e interesse per il lettore. In particolare, i test sono significativi quando verificano proprietà che impattano direttamente l'affidabilità del sistema, come coerenza dei dati serializzati, correttezza dei vincoli di sequenza dei messaggi, gestione dei limiti di frequenza e stabilità del provisioning su backend reale.

6.1 Analisi delle suites di test

6.1.1 Moduli di dominio e consistenza dati

`tst_models` verifica la validazione dei principali DTO (utente, canale, messaggio, sessione, membership, configurazione) e la correttezza dei round-trip JSON. L'interesse di questa suite è alto perché previene errori strutturali a monte della pipeline applicativa: se i modelli non sono coerenti, i moduli superiori diventano fragili.

`tst_errormodel` valida la semantica degli errori applicativi (`hasError()`, serializzazione testuale, mappatura dei codici), garantendo che logging e propagazione degli errori siano uniformi.

6.1.2 Moduli di controllo del flusso

`tst_ratelimiter` verifica il comportamento del token bucket (capacità, refill, disponibilità token), cioè il meccanismo che protegge il sistema da burst non controllati.

`tst_recentmessagecache` e `tst_messageprocessor` validano deduplicazione, filtro per contesto e gestione della sequenza dei messaggi, aspetti essenziali per evitare doppie elaborazioni e inconsistenze nel client.

`tst_recoverymanager` e `tst_messagequeryservice` verificano la logica di recupero storico, con particolare attenzione alla ricostruzione dei gap e alla query dei messaggi mancanti.

6.1.3 Moduli applicativi

`tst_appcommclient` copre la coerenza del comportamento del client nel ciclo di vita operativo (configurazione, inizializzazione, gestione canale e messaggi).

`tst_appcommserver` analizza la facciata server: provisioning logico, gestione canali, utenti, membership e operazioni di broadcasting. Questa suite è centrale perché esercita i casi d'uso amministrativi che abilitano l'intero sistema di comunicazione.

6.1.4 Integrazione con infrastruttura reale

`tst_appwritesdk` è il test di integrazione più significativo: valida end-to-end il flusso incrementale su Appwrite reale (create/list/delete su utente, database, collezione, attributi, documenti e indice), inclusa la fase di cleanup. Richiede `APPWRITE_ENDPOINT`, `APPWRITE_PROJECT_ID`, `APPWRITE_API_KEY`.

`tst_realtime` e `tst_garbagecollector` coprono in parte la componente realtime/cleanup; alcuni casi restano condizionati dalla disponibilità di mocking specifico, per mantenere il test deterministico.

6.2 Risultati

L'esecuzione della campagna di test, produce 13 suites registrate da CTest:

L'esito complessivo dell'ultima esecuzione è positivo, con tutte le suites registrate completate con successo. Il risultato è rilevante in quanto include sia test puramente logici, sia test che validano l'integrazione in un'istanza di Appwrite realmente esistente, confermando che le astrazioni implementate dalla libreria sono operative anche oltre il contesto unitario.

ID	Nome	Requisito	Esito
T-01	tst_appwritesdk	R-01	<i>Passato</i>
T-02	tst_models	R-02/03/07/08	<i>Passato</i>
T-03	tst_messageprocessor	R-04	<i>Passato</i>
T-04	tst_recoverymanager	R-05	<i>Passato</i>
T-05	tst_recentmessagecache	R-06	<i>Passato</i>
T-06	tst_errormodel	R-08	<i>Passato</i>
T-07	tst_realtime	R-09	<i>Passato</i>
T-08	tst_ratelimiter	R-09	<i>Passato</i>
T-09	tst_appcommserver	R-10	<i>Passato</i>
T-10	tst_appcommclient	R-10	<i>Passato</i>
T-11	tst_messagequeryservice	R-11	<i>Passato</i>
T-12	tst_membershipservice	R-11	<i>Passato</i>
T-13	tst_garbagecollector	R-12	<i>Passato</i>

Tabella 6.1: Risultati delle test suites

6.3 Conclusioni

La validazione mostra come l'architettura proposta sia supportata da un set di test articolato per responsabilità e allineamento alle aree tecniche più critiche del progetto. Le suites sui modelli e sul processing garantiscono consistenza del dato e correttezza del flusso; le suites client/server verificano i percorsi operativi principali mentre il test di integrazione su Appwrite fornisce una prova concreta di funzionamento end-to-end.

Dal punto di vista metodologico, il valore della campagna non sta solamente nel numero di test, bensì anche nella qualità del loro posizionamento; i controlli sono concentrati nei punti in cui un difetto avrebbe maggior impatto sistematico. Questo rende i risultati utili sia come evidenza di correttezza attuale, sia come base di regressione per sviluppi futuri del progetto.

Capitolo 7

Risultati

Questo capitolo illustra e analizza i risultati ottenuti a seguito dell'implementazione del Proof of Concept e dell'esecuzione delle test suites descritte nel capitolo precedente. L'obiettivo principale di questa fase è valutare in modo oggettivo l'efficacia della libreria appcomm rispetto ai requisiti funzionali e architetturali delineati all'inizio del progetto. L'analisi che segue non si limita a confermare il superamento dei test logici, ma esplora il comportamento del sistema nel suo complesso, osservando l'interazione tra il frontend, il middleware e il backend in condizioni operative reali.

La valutazione si concentra su tre assi principali: la correttezza architetturale, l'affidabilità della comunicazione e le prestazioni del sistema. Verranno discussi gli esiti relativi al provisioning automatico dell'infrastruttura, dimostrando come il backend sia in grado di auto-configurare il database e le collezioni necessarie senza intervento manuale. Successivamente verrà analizzata la reattività del sistema in tempo reale, valutando l'impatto della latenza introdotta dal paradigma BaaS, rispetto a soluzioni tradizionali. Infine si esaminerà la robustezza del meccanismo di recovery durante scenari di disconnessione o perdita di pacchetti, verificando la capacità del sistema di mantenere uno stato coerente. Attraverso questa disamina il capitolo funge da ponte tra la validazione puramente tecnica e la valutazione dell'usabilità della soluzione per future integrazioni.

7.1 Valutazione architetturale

L'implementazione del Proof of Concept ha confermato la validità dell'architettura proposta. L'utilizzo di Appwrite come proxy intermedio ha permesso di stabilire una comunicazione bidirezionale tra il client ed il backend, aggirando completamente le restrizioni imposte dalle tradizionali reti private, senza la necessità di esporre porte o aprire servizi che siano potenziali vettori di insicurezze.

7.2 Validazione del Proof of Concept

Il PoC ha dimostrato con successo l'interoperabilità tra un frontend compilato per il web, ed un backend nativo, condividendo la medesima base di codice per la logica di comunicazione. In particolare si è osservato che:

- La capacità del backend di configurare autonomamente l'ambiente Appwrite (creazione di database, collezioni e indici) riduce drasticamente i tempi di deployment, soddisfacendo il requisito di "bootstrap", delineato in fase di analisi.
- L'integrazione di un WebSocket ha garantito i tempi di latenza minimi nella ricezione dei messaggi, confermando che l'approccio *push-based* di Appwrite è una valida alternativa al polling HTTP tradizionale.

7.3 Prestazioni e limiti riscontrati

Sebbene il sistema risponda in modo eccellente in scenari di traffico normale, i test hanno evidenziato che la delega del traffico ad un servizio BaaS esterno introduce una latenza di base inevitabile rispetto ad una connessione socket diretta in rete locale. Tuttavia l'implementazione del Rate Limiter ha dimostrato di poter mitigare efficacemente i picchi di richieste, proteggendo il backend e garantendo il rispetto delle quote API imposte dal server Appwrite.

7.4 Manuale d'uso

Il manuale d'uso è riportato nei rispettivi allegati, in coda al documento, così da poter essere utilizzato come guida rapida durante l'installazione, la configurazione e la gestione quotidiana del sistema. Sono presenti la guida utente per il backend user (A) e per il frontend user (B).

Capitolo 8

Conclusioni

Giunti al termine di questo percorso di progettazione, sviluppo e validazione, il presente capitolo ha lo scopo di tirare le somme sull'intero lavoro di semestre. Il progetto ha affrontato una delle problematiche più ricorrenti e complesse nello sviluppo di applicazioni distribuite moderne: la necessità di instaurare una comunicazione bidirezionale sicura e affidabile tra nodi operanti in reti private, superando rigide policy di sicurezza di firewall e NAT. Invece di ricorrere a complessi tunnel VPN oppure a configurazioni di rete fragili, si è scelto di esplorare e validare un approccio basato sull'utilizzo di una piattaforma Backend-as-a-Service come middleware di comunicazione.

Nelle prossime sezioni verranno sintetizzati i traguardi raggiunti, confrontando gli esiti ottenuti con gli obiettivi originari prefissati nella fase di analisi dei requisiti. Si rifletterà sul valore architetturale del paradigma adottato, che sposta la complessità dell'infrastruttura verso un layer di cloud gestito, alleggerendo le responsabilità del server applicativo. Verranno inoltre discusse le implicazioni tecnologiche derivanti dall'utilizzo congiunto di C++, del framework Qt e della compilazione WebAssembly, una sinergia che ha garantito un'altissima riusabilità del codice. Infine, partendo dai limiti e dalle osservazioni emerse durante lo sviluppo, verranno delineati i possibili sviluppi futuri che potrebbero estendere ulteriormente le capacità e la sicurezza della libreria realizzata.

8.1 Sintesi del lavoro

Spostando la complessità infrastrutturale su Appwrite, si è ottenuto un sistema dove il backend può operare in totale sicurezza dall'interno di una rete privata aggirando i firewall. La libreria è stata strutturata con un'architettura fortemente modulare che separa le responsabilità tra il core di comunicazione (appcomm), l'applicazione client con interfaccia Qt/QML (appfrontend) e l'applicazione server headless (backend).

La scelta di utilizzare C++ e Qt, unita a WebAssembly per il deployment su browser, si è rivelata estremamente efficace, garantendo una completa compatibilità cross-platform per Linux, macOS e Windows. Lo sviluppo ha portato alla realizzazione di un SDK di basso livello per Appwrite stabile e funzionante, capace di astrarre le richieste HTTP, la gestione delle sessioni (sia anonime che tramite email), le operazioni CRUD sui documenti e la gestione amministrativa degli utenti. L'integrazione del frontend è stata inoltre agevolata dall'uso di macro come Q_ENUM per abilitare dinamicamente le funzionalità dell'interfaccia utente QML.

8.2 Sviluppi futuri

Dal momento che l'architettura di base, comprensiva di astrazione dei canali, meccanismi di recovery, comunicazione realtime e orchestrazione del backend, ha raggiunto uno stato di maturità ed è stata pienamente validata, le future direzioni di sviluppo si concentrano sull'espansione delle funzionalità, sulla sicurezza avanzata e sull'ottimizzazione delle performance in ambienti vincolati.

I prossimi passi per l'evoluzione del framework appcomm includono:

- Attualmente, la sicurezza del sistema è garantita dal protocollo TLS in transito e dai rigorosi controlli di accesso (Document-Level Security) offerti da Appwrite. Tuttavia, i payload JSON transitano in chiaro attraverso il middleware. L'implementazione di uno strato di crittografia asimmetrica (ad esempio tramite curve ellittiche, ECC) integrato nativamente nella libreria C++ permetterebbe di cifrare i messaggi sul client prima dell'invio. In questo scenario, Appwrite agirebbe come un relay puramente Zero-Knowledge, impossibilitato a decifrare il contenuto dei dati instradati.
- Sebbene il sistema integri già un modulo RecentMessageCache in memoria per far fronte a brevi disconnessioni, un'evoluzione significativa prevederebbe la persistenza dello stato a lungo termine. Sfruttando le astrazioni di Qt (ad esempio QSql mappato su IndexedDB per il target WebAssembly), la libreria potrebbe salvare i messaggi in locale, garantendo la consultazione della cronologia in assenza di rete e implementando una coda di sincronizzazione automatica non appena il dispositivo torna online.
- L'attuale implementazione è focalizzata sullo scambio rapido di payload JSON testuali strutturati. Estendere l'SDK per supportare le API Storage di Appwrite abiliterebbe lo scambio di allegati e file binari. Questa evoluzione richiederebbe l'implementazione di meccanismi di chunking e upload asincrono in C++, per garantire che il caricamento di file di grandi dimensioni non blocchi il thread principale, specialmente cruciale nel contesto del Single Thread di WebAssembly.
- Allo stato attuale, la gestione della serializzazione JSON, il parsing e il meccanismo di limitazione del traffico avvengono sul thread principale dell'applicazione. In un'ottica di scalabilità verso scenari ad altissima frequenza di messaggi, la libreria potrebbe essere ottimizzata spostando il carico computazionale del parsing e della cifratura su thread separati (utilizzando QtConcurrent) o demandando l'onere ai Web Workers nel caso specifico del frontend compilato in WebAssembly, migliorando la fluidità della UI.

8.3 Considerazioni finali

In conclusione, il progetto Proxy ha dimostrato come l'integrazione di tecnologie apparentemente distanti; il calcolo nativo in C++, la flessibilità del framework Qt e la modernità dei servizi Backend-as-a-Service come Appwrite, possa convergere in una soluzione funzionale. La sfida iniziale di garantire una comunicazione bidirezionale fluida attraverso reti isolate è stata superata non solo attraverso l'implementazione tecnica della libreria appcomm, ma anche attraverso la definizione di un'architettura capace di scalare e adattarsi a diversi scenari d'uso, dal desktop al web tramite WebAssembly.

Il successo del Proof of Concept e la solidità dell'infrastruttura di test e integrazione continua confermano che l'approccio adottato è pronto per evolversi da un prototipo accademico a uno strumento di sviluppo professionale. Al di là dei risultati tecnici, questo lavoro ha rappresentato un importante percorso di crescita nella gestione di sistemi distribuiti e nella progettazione di API asincrone, lasciando in eredità un framework solido su cui costruire le prossime innovazioni nel campo delle comunicazioni cross-network.

Glossario

- Appwrite** Una piattaforma open-source per lo sviluppo di applicazioni web e mobile, che fornisce una serie di servizi backend come autenticazione, database, storage e funzioni serverless. 5
- FIFO** First In First Out, modalità di gestione di una coda di dati, nella quale i primi elementi aggiunti sono anche i primi ad essere rimossi. 18
- Firewall** Sistema hardware o software che controlla e filtra il traffico di rete in ingresso e in uscita secondo regole di sicurezza. i, 7, 8
- HTTP** Hypertext Transfer Protocol. Protocollo applicativo standard del Web usato per lo scambio di richieste e risposte tra client e server. 14, 21, 26
- HTTPS** Hypertext Transfer Protocol Secure. Versione sicura di HTTP che protegge la comunicazione tramite crittografia TLS. 26
- Inbound** Comunicazione di rete direzionata dall'esterno verso la parte interna di una rete LAN. 7
- JWT** JSON Web Token. Formato compatto per rappresentare in modo sicuro informazioni e claim tra parti, spesso usato per autenticazione e autorizzazione. 9, 15
- LAN** Local Area Network. Rete locale che collega dispositivi in un'area geografica limitata, come casa, ufficio o campus. 8
- LRU** Least Recently Used, modalità di eviction di una memoria volatile, dove, una volta raggiunta la capacità massima, il dato aggiunto meno di recente viene espulso, in favore di uno nuovo. 18
- MQTT** Message Queuing Telemetry Transport. Protocollo di messaggistica leggero basato su modello publish/subscribe, diffuso in ambito IoT. 8, 9
- NAT** Network Address Translation. Una tecnica usata dai router per modificare gli indirizzi IP dei pacchetti in transito, permettendo a più dispositivi di condividere un singolo indirizzo IP pubblico. 7, 8
- Outbound** Comunicazione di rete direzionata verso l'esterno di una rete locale LAN. 7
- QML** Qt Modeling Language. Linguaggio dichiarativo di Qt per definire interfacce utente dinamiche e componenti grafici. 4
- REST** Representational State Transfer. Stile architetturale per la progettazione di API web basato su risorse e operazioni standard HTTP. 8, 14, 15, 21

SDK Software Development Kit. Insieme di librerie, strumenti e documentazione per sviluppare applicazioni su una piattaforma o servizio specifico. i, 11, 14–16, 19, 21, 28, 33, 37, 39

Throughput Capacità di trasmissione effettiva che viene usata in una certa unità di tempo. i, 19

TTL Time to Live, intervallo di tempo della durata di validità di un dato prima della sua scadenza o eliminazione automatica. È solitamente espresso in secondi ed è utilizzato per gestire la persistenza temporanea di elementi come messaggi, sessioni o risorse di cache. 13, 27

WAN Wide Area Network. Rete geografica estesa che collega tra loro più reti locali su grandi distanze. 8

WebAssembly Un formato di istruzioni binario portabile, progettato per consentire l'esecuzione di codice ad alte prestazioni all'interno dei browser web. 5

WebSocket Protocollo di comunicazione bidirezionale e full-duplex su una singola connessione TCP, usato per comunicazioni realtime tra client e server. 8, 25, 26

Sitografia

- [1] The Qt Company. *Qt 6 Documentation*. 2026. URL: <https://doc.qt.io/qt-6/> (visitato il 18/02/2026).
- [2] Appwrite Team. *Appwrite Documentation - Open-Source Backend Server*. 2026. URL: <https://appwrite.io/docs> (visitato il 18/02/2026).
- [3] Geeks for Geeks. *Token Bucket Algorithm*. 2026. URL: <https://geeksforgeeks.org/computer-networks/token-bucket-algorithm/> (visitato il 16/03/2026).
- [4] The Qt Company. *Qt for WebAssembly*. 2026. URL: <https://doc.qt.io/qt-6/wasm.html> (visitato il 02/05/2026).

Appendice A

Guida utente - Backend

La presente guida descrive l'utilizzo dell'eseguibile binaria backend, il quale fornisce funzioni amministrative per Appwrite, come un servizio "echo" per il Proof of Concept.

A.1 Prerequisiti

- progetto compilato con target backend disponibile;
- istanza Appwrite raggiungibile;
- API key Appwrite con privilegi amministrativi (database, collections, documents, users).

A.2 Configurazione iniziale

Il backend salva la configurazione nel file:

```
~/ .proxy-backend-config.json
```

È possibile cambiare percorso con la variabile d'ambiente:

```
export PROXY_BACKEND_CONFIG_PATH="/path/to/backend-config.json"
```

Configurazione (crea/valida anche la struttura su Appwrite):

```
backend configure <endpoint> <projectId> <apiKey> <databaseId> \
    [messagesCollection] \
    [membersCollection] \
    [channelsCollection] \
    [sessionsCollection]
```

Esempio:

```
backend configure https://cloud.appwrite.io/v1 my-project my-api-key proxydb
```

A.3 Avvio del servizio

Per avviare il servizio long-running (echo server):


```
backend start
```

Alias equivalenti:

```
backend serve  
backend echo-service
```

A.4 Comandi amministrativi

A.4.1 Utenti

```
backend users list  
backend users add <email> <password> [name]  
backend users remove <userId>
```

A.4.2 Canali

```
backend channels list  
backend channels add <name>  
backend channels remove <topicId>
```

A.4.3 Membri del canale

```
backend members list <topicId>  
backend members add <topicId> <userId> [displayName]  
backend members remove <topicId> <userId>
```

A.4.4 Sessioni

```
backend sessions list  
backend sessions add <topicId> <userId> [userId ...]  
backend sessions close <sessionId>
```

A.4.5 Messaggi

```
backend messages list <topicId> [limit]  
backend messages find <topicId> <messageId> [limit]  
backend messages add <topicId> <senderId> <payloadJsonOrText>  
backend messages remove <messageId>
```

Nota: quando `payloadJsonOrText` non è JSON valido, il backend lo salva come testo nel campo `text`.

A.5 Flusso operativo consigliato

1. Eseguire `backend configure` ... una sola volta per ambiente.
2. Creare utenti (`users add`) e canali (`channels add`).
3. Associare utenti ai canali (`members add`).
4. Aprire una sessione logica (`sessions add`) e pubblicare messaggi (`messages add`).
5. Avviare `backend start` quando si vuole il comportamento di echo automatico.

A.6 Output e gestione errori

- i risultati delle operazioni vengono stampati su `stdout` in formato JSON;
- gli errori vengono stampati su `stderr` con codice di uscita diverso da zero;
- se la configurazione non esiste, il comando richiede esplicitamente di eseguire `backend configure`.

Appendice B

Guida utente - Frontend

La presente guida descrive l'utilizzo dell'applicazione frontend del Proof of Concept, realizzata con Qt/QML. Il frontend permette di interagire con la libreria appcomm tramite un'interfaccia grafica minimale, consentendo il login, l'accesso alla chat, l'invio e la ricezione di messaggi e il logout.

B.1 Prerequisiti

- progetto compilato con target frontend disponibile;
- istanza Appwrite raggiungibile e già configurata tramite backend;
- servizio backend avviato, se si vuole osservare il comportamento di risposta automatica in modalità echo;
- utente Appwrite esistente, nel caso di login tramite email;
- membership associata all'utente, cioè un topic assegnato tramite il backend.

Per poter accedere correttamente alla schermata di chat, l'utente deve essere associato a un topic. In caso contrario il client non è in grado di determinare il contesto della conversazione e l'accesso alla chat non viene completato.

B.2 Configurazione iniziale

Il frontend salva la configurazione nel file:

```
~/ .proxy-frontend-config.json
```

È possibile cambiare percorso con la variabile d'ambiente:

```
export PROXY_FRONTEND_CONFIG_PATH="/path/to/frontend-config.json"
```

Configurazione:

```
appfrontend configure <endpoint> <projectId> <databaseId> \  
                    [messagesCollection] \  
                    [membersCollection] \  
                    [incomingMessagesCollection]
```

I parametri richiesti sono:

endpoint endpoint Appwrite, ad esempio `https://fra.cloud.appwrite.io/v1`;

projectId identificativo del progetto Appwrite;

databaseId identificativo del database Appwrite;

messagesCollection collection contenente i messaggi finali. Se non viene specificata, viene usato il valore predefinito `messages`;

membersCollection collection contenente le associazioni tra utenti e canali. Se non viene specificata, viene usato il valore predefinito `members`;

incomingMessagesCollection collection usata per i messaggi in ingresso. Se non viene specificata, viene usato il valore predefinito `pendingmessages`.

Esempio:

```
appfrontend configure https://fra.cloud.appwrite.io/v1 my-project proxydb \
  messages members pendingmessages
```

Il comando genera un file JSON con una struttura simile alla seguente:

```
{
  "endpoint": "https://fra.cloud.appwrite.io/v1",
  "projectId": "my-project",
  "databaseId": "proxydb",
  "messagesCollectionId": "messages",
  "membersCollectionId": "members",
  "incomingMessagesCollectionId": "pendingmessages"
}
```

B.3 Avvio dell'applicazione

Dopo la compilazione del progetto, il frontend può essere avviato eseguendo il relativo binario generato dalla build Qt.

```
frontend
```

Il nome effettivo dell'eseguibile può variare in base alla configurazione del progetto e dell'ambiente di build utilizzato.

B.4 Schermata iniziale

All'avvio, l'applicazione mostra una schermata di scelta del metodo di accesso. Da questa schermata è possibile:

- accedere come guest;
- accedere tramite credenziali email;

B.5 Login tramite email

Per accedere tramite email è necessario seguire i seguenti punti:

1. Selezionare l'opzione di login tramite email.
2. Inserire email e password.
3. Confermare il login.

Se le credenziali sono corrette e l'utente possiede una membership valida, il client recupera il topic associato, carica gli ultimi 16 messaggi disponibili e apre la schermata di chat.

B.6 Login come guest

Il frontend permette anche di tentare l'accesso tramite sessione guest.

1. Selezionare l'opzione di login guest.
2. Attendere la risposta del client.

Nel PoC l'accesso guest viene utilizzato anche per validare il caso di utente non autorizzato. Se il guest non dispone dei permessi necessari o non possiede una membership associata, l'applicazione mostra una schermata dedicata di accesso negato. Da questa schermata è possibile tornare alla pagina iniziale di login.

B.7 Utilizzo della chat

Dopo un login completato correttamente, viene mostrata la schermata di chat. La schermata contiene:

- il topic corrente assegnato all'utente;
- lo stato della connessione realtime;
- la lista dei 16 messaggi caricati con estetica da app di messaggistica;
- il campo di input per inviare nuovi messaggi;
- il bottone per inviare il testo inserito nel campo di input;
- il pulsante di logout.

All'apertura della chat, il client carica gli ultimi 16 messaggi disponibili per il topic assegnato. I messaggi vengono mostrati nell'interfaccia distinguendo quelli inviati dall'utente corrente da quelli ricevuti da altri utenti o generati dal backend.

B.8 Invio dei messaggi

Per inviare un messaggio:

1. scrivere il testo nel campo di input;
2. premere il pulsante di invio ("send").

Il messaggio viene inviato alla collection dei messaggi in attesa. Successivamente il backend lo processa, assegna il numero di sequenza e pubblica il messaggio definitivo nella collection dei messaggi. Se il servizio echo è attivo, viene generata anche una risposta automatica.

B.9 Ricezione dei messaggi

Il frontend riceve i nuovi messaggi tramite la connessione realtime aperta da `AppcommClient`. Quando un nuovo messaggio viene pubblicato su Appwrite, il client riceve l'evento, lo elabora e aggiorna automaticamente la lista mostrata nella chat.

B.10 Logout

Per terminare la sessione è possibile premere il pulsante di logout nella schermata di chat.

1. Premere Logout.
2. Il client chiude la sessione corrente e disconnette il realtime.
3. Dopo una breve attesa di 2 secondi, l'applicazione torna alla schermata iniziale.

B.11 Errori comuni

- `No membership found for current user.`: l'utente esiste, ma non è associato ad alcun topic.
- `Invalid membership document.`: il documento di membership esiste, ma non contiene dati validi.
- `Guest access denied`: l'accesso guest non dispone dei permessi necessari.
- `Rate limit exceeded.`: sono stati inviati troppi messaggi in un intervallo di tempo troppo breve.

B.12 Flusso operativo consigliato

1. Configurare Appwrite tramite il backend.
2. Creare almeno un utente e un topic.
3. Associare l'utente al topic tramite membership.
4. Avviare il servizio backend in modalità echo, se richiesto dalla dimostrazione.
5. Avviare il frontend.
6. Effettuare il login con l'utente configurato.
7. Inviare messaggi e verificare la ricezione degli aggiornamenti in realtime.
8. Eseguire il logout per tornare alla schermata di login iniziale.

B.13 Avvio della versione WebAssembly

Dopo la compilazione del target WebAssembly, i file generati per l'esecuzione nel browser si trovano nella directory `build-wasm/frontend`. È possibile servirli tramite un semplice server HTTP locale:

```
cd build-wasm/frontend  
python -m http.server 8000
```

L'applicazione è quindi raggiungibile da browser all'indirizzo:

```
http://localhost:8000/appfrontend.html
```